Dart

Complete Programming Guide

With detailed explanations for every concept, operator, method, and pattern

Ch 1-9: Core Language | Ch 10-16: Advanced | Ch 17-22: Concurrency & Libraries | Ch 23-27: Collections

Ch 1 Basics & Syntax
Ch 2 Null Safety
Ch 3 Operators
Ch 4 Strings
Ch 5 Control Flow & Loops
Ch 6 Functions
Ch 7 OOP: Classes & Objects
Ch 8 Async Programming
Ch 9 Generics & Iterables
Ch 10 Enums
Ch 11 Extensions
Ch 12 Typedefs & Callable Classes
Ch 13 Patterns, Records & Sealed Classes
Ch 14 Advanced OOP
Ch 15 Constructors (Advanced)
Ch 16 Annotations & Metadata
Ch 17 Error & Exception Handling
Ch 18 Isolates & Concurrency
Ch 19 Dart Core Libraries
Ch 20 StringBuffer & Regex
Ch 21 Advanced Async: Futures & Streams
Ch 22 Generics (Advanced)
Ch 23 List Methods (Complete)
Ch 24 Set Methods (Complete)
Ch 25 Map Methods (Complete)
Ch 26 Collection-if/for & Spreads
Ch 27 Quick Comparison Table

Chapter 1 -- Basics & Syntax

Dart is a strongly-typed, object-oriented language built by Google. It is the foundation of Flutter (cross-platform UI) and can compile to native machine code (AOT) for production or use JIT compilation for fast development cycles. Every Dart program must have a main() function as its entry point.

1.1 Hello World & Program Structure

Every Dart program starts from the main() function. The void return type means the function returns nothing. Statements end with a semicolon. The print() function outputs to the console and automatically adds a newline.

```
// Every Dart program starts here:
void main() {
  print('Hello, Dart!');   // Outputs: Hello, Dart!
  print(2 + 3);            // Outputs: 5
  print('Sum: ${2 + 3}'); // String interpolation: Sum: 5
}
```

>> Output:

```
Hello, Dart!
5
Sum: 5
```

1.2 Variables: var, final, const

Dart gives you three ways to declare a variable. 'var' infers the type but lets you reassign. 'final' means the variable can only be set once -- its value is locked after first assignment. 'const' means the value must be known at compile time and is deeply immutable. Understanding this distinction prevents many common bugs.

```
// var -- type inferred, value can change later:
var name = 'Alice';    // Dart infers type as String
name = 'Bob';          // OK -- reassignment allowed
// name = 42;          // Error! Cannot assign int to String

// final -- assigned once at RUNTIME, then locked:
final age = DateTime.now().year - 1995;  // computed at runtime
// age = 30;           // Error! final variable cannot be reassigned

// const -- must be known at COMPILE TIME:
const pi = 3.14159;
const maxItems = 100;
// pi = 3.0;           // Error! const cannot be changed

// Explicit type annotations (when you want to be clear):
String city = 'NYC';
int count = 0;
```

```
double price = 9.99;
bool isActive = true;
```

final vs const

Use final for values that are computed at runtime (e.g. user input, DateTime.now()). Use const only for values known before the program runs (e.g. 3.14, 'hello'). In Flutter, const widgets are more efficient because they are never rebuilt.

1.3 Data Types Overview

Dart is strongly typed -- every variable has a type. The compiler checks types at compile time, catching errors before you even run your code. The main built-in types are listed below. Everything in Dart is an object, even primitive numbers.

Type	Example Values	Description
`int`	`42, -7, 0, 0xFF`	Whole numbers (64-bit). No size limit on web.
`double`	`3.14, -0.5, 1.0e10`	Floating-point numbers (64-bit IEEE 754)
`num`	`42 or 3.14`	Parent of int and double -- accepts either
`String`	`'hello', "world", r'raw'`	Sequence of UTF-16 code units
`bool`	`true, false`	Boolean -- ONLY true or false, no truthy values
`List<T>`	`[1, 2, 3]`	Ordered, indexed, growable sequence
`Set<T>`	`{1, 2, 3}`	Unordered collection of unique elements
`Map<K,V>`	`{'key': 'value'}`	Collection of key-value pairs
`Null`	`null`	The absence of a value (its own type in Dart)
`dynamic`	`dynamic x = anything`	Disables static type checking -- use sparingly
`Object`	`Object o = 42`	Root base type for all Dart objects
`Symbol`	`#mySymbol`	Represents identifiers as objects

1.4 Type Conversion

Dart does NOT do implicit type conversion. You cannot add an int to a String without explicitly converting one. Each numeric type has dedicated methods for converting to other types. The tryParse variants are safer -- they return null on failure instead of throwing.

```
// int and double conversions:
int a = 42;
double d = a.toDouble();   // 42.0  -- int becomes double
int b = d.toInt();         // 42    -- truncates (does NOT round)
int c = 3.9.round();       // 4     -- use round() to round
int fl = 3.9.floor();      // 3     -- floor always rounds down

// String to number:
```

```
int parsed  = int.parse('42');           // 42  -- throws on failure
double dp   = double.parse('3.14');      // 3.14
int? safe   = int.tryParse('abc');       // null -- no exception
int? ok     = int.tryParse('99');        // 99

// Number to String:
String s1 = 42.toString();              // '42'
String s2 = 3.14159.toStringAsFixed(2); // '3.14' -- 2 decimals
String s3 = 255.toRadixString(16);      // 'ff'   -- hex string
```

1.5 String Interpolation & Literals

Dart's string interpolation uses the dollar sign ($) to embed expressions directly inside strings. Use ${} for any expression, and just $ for simple variable names. This is cleaner and faster than concatenation with the + operator.

```
String name = 'Alice';
int age = 30;

// Simple interpolation -- $variable:
print('Hello, $name!');                  // Hello, Alice!

// Expression interpolation -- ${expression}:
print('Next year: ${age + 1}');           // Next year: 31
print('Upper: ${name.toUpperCase()}');    // Upper: ALICE

// Multi-line string (triple quotes):
String poem = '''
Line one
Line two
Line three''';

// Raw string -- backslashes are literal, no escape processing:
String path  = r'C:\Users\file.txt';   // prints as-is
String regex = r'\d+\.\d+';            // useful for regex patterns

// Adjacent literals auto-concatenate (no + needed):
String long = 'Hello '
    'World';          // 'Hello World'
```

Chapter 2 -- Null Safety

Sound null safety (introduced in Dart 2.12) is one of Dart's most important features. It eliminates an entire class of bugs -- the dreaded 'null reference error' -- at compile time. The rule is simple: variables cannot be null unless you explicitly say so. This forces you to handle null cases intentionally.

2.1 Nullable vs Non-Nullable Types

By default, every variable in Dart is NON-NULLABLE -- the compiler guarantees it can never be null. To allow null, you add a question mark (?) after the type. This contract is enforced at compile time, not runtime, so you catch problems early.

```
// Non-nullable (default) -- CANNOT be null:
String name = 'Alice';
// name = null;         // COMPILE ERROR -- null not allowed

// Nullable -- add ? to allow null:
String? nickname = null;   // OK -- explicitly nullable
nickname = 'Ali';          // Also OK -- can assign a value
int? score;                // Also null -- uninitialized nullable

// The compiler tracks nullability:
String? val = maybeGetValue();
print(val.length);         // COMPILE ERROR -- val might be null!

// You must check first:
if (val != null) {
  print(val.length);       // OK -- compiler knows val is not null here
}
```

2.2 Null-Aware Operators

These operators provide compact, safe ways to work with nullable values. They eliminate the need for verbose null-check if-statements and make null-handling code readable. Each operator has a specific purpose -- learn them all as they appear constantly in real Dart code.

```
String? name = null;

// ?? (null coalescing) -- use right side if left is null:
String display = name ?? 'Anonymous';   // 'Anonymous'
print(display.length);                  // 9

// ??= (null-aware assignment) -- assign only if currently null:
name ??= 'Default';   // name was null, now 'Default'
```

```
name ??= 'Other';      // name is 'Default', no change -- skipped
print(name);           // 'Default'

// ?. (null-safe member access) -- skips call if object is null:
String? city = null;
print(city?.toUpperCase());   // null (no exception thrown)
print(city?.length ?? 0);     // 0 (combined with ??)

// ?[] (null-safe index) -- safe access on nullable collections:
List<int>? nums = null;
print(nums?[0]);              // null (no RangeError)

// ! (null assertion) -- tells Dart 'I know this is not null':
String nonNull = name!;   // Safe here -- name is 'Default'
```

WARNING: The ! Operator

Using name! when name is actually null throws a runtime error: 'Null check operator used on a null value'. Only use ! when you are 100% certain the value is not null. It defeats null safety, so prefer other operators.

2.3 Late Variables

'late' is for non-nullable variables that you cannot initialize immediately -- they will definitely be set before being read, but the compiler cannot verify this automatically. Common in class fields, dependency injection, and lazy initialization.

```
// late -- non-nullable, but initialized later:
late String description;
// Using description here would throw LateInitializationError
description = 'Initialized now';
print(description);   // Safe -- already initialized

// late final -- lazy initialization (only computed when first used):
late final String expensive = computeExpensiveValue();
// computeExpensiveValue() is NOT called until you access 'expensive'

// Common in Flutter classes that need initState:
class MyWidget extends StatefulWidget {
  @override State<MyWidget> createState() => _MyWidgetState();
}
class _MyWidgetState extends State<MyWidget> {
  late final AnimationController _controller; // set in initState

  @override void initState() {
    super.initState();
    _controller = AnimationController(vsync: this); // safe!
  }
```

```
}
```

2.4 Common Null Safety Patterns

These patterns appear constantly in production Dart code. Learn each one -- they solve specific null-handling scenarios more elegantly than verbose if-checks.

```
// Pattern 1: Early return guard -- handle null early and exit:
void greet(String? name) {
  if (name == null) return;   // exits immediately
  print('Hello, $name!');    // name is String (non-null) here
}

// Pattern 2: Null coalescing chain -- nested nullable access:
String display = user?.profile?.displayName ?? 'Guest';
// Safe even if user is null, or profile is null

// Pattern 3: Conditional ternary:
String? val = getOptionalValue();
String result = val != null ? val.toUpperCase() : 'NONE';

// Pattern 4: Required named parameters (always non-null):
void createUser({required String name, int age = 18}) {
  print('$name, age $age');
}
createUser(name: 'Alice');         // name must be provided
createUser(name: 'Bob', age: 25);

// Pattern 5: Null check with assignment (walrus-like):
final value = expensiveOperation();
if (value != null) {
  process(value);   // value promoted to non-null inside block
}
```

Chapter 3 -- Operators & Expressions

Operators are special symbols that perform operations on values. Dart has a rich set of operators -- many familiar from other languages, plus some unique ones like cascade (..), spread (...), and null-aware operators. Understanding operator precedence (which operations run first) prevents subtle bugs.

3.1 Arithmetic Operators

Arithmetic operators work on numbers. One important difference from many languages: in Dart, the / operator ALWAYS returns a double, even when dividing two integers. Use ~/ for integer division that returns an int (truncates toward zero).

```
// Basic arithmetic:
print(10 + 3);    // 13   -- addition
print(10 - 3);    // 7    -- subtraction
print(10 * 3);    // 30   -- multiplication
print(10 / 3);    // 3.333... -- TRUE division, ALWAYS returns double!
print(10 ~/ 3);   // 3    -- integer division (floor division)
print(10 % 3);    // 1    -- modulo: remainder after integer division

// Unary operators -- pre/post increment/decrement:
int a = 5;
print(a++);  // 5  -- post-increment: returns THEN adds (a becomes 6)
print(a);    // 6
print(++a);  // 7  -- pre-increment: adds THEN returns
print(a--);  // 7  -- post-decrement: returns THEN subtracts
print(--a);  // 5  -- pre-decrement

// Math functions from dart:math:
import 'dart:math';
print(pow(2, 8));   // 256.0 -- 2 to the power of 8
print(sqrt(144));   // 12.0  -- square root
```

3.2 Comparison & Equality

Dart uses == for value equality (checks if values are equal) and identical() to check if two variables point to the same object in memory. For primitive types like String and int, == usually behaves as expected. For collections like List, == checks identity by default, not content.

```
// Comparison operators -- all return bool:
print(5 == 5);    // true   -- equal value
print(5 != 3);    // true   -- not equal
print(5 > 3);     // true   -- greater than
print(5 >= 5);    // true   -- greater than or equal
print(3 < 5);     // true   -- less than
```

```
print(3 <= 5);    // true   -- less than or equal

// == checks VALUE equality:
String a = 'hello';
String b = 'hello';
print(a == b);           // true -- same value
print(identical(a, b));  // true -- Dart interns small strings

// Lists: == checks IDENTITY by default, not content!
List x = [1, 2, 3];
List y = [1, 2, 3];
print(x == y);           // false -- different objects in memory
print(identical(x, y));  // false
// Use package:collection's listEquals() for deep comparison
```

3.3 Logical Operators

Logical operators combine boolean expressions. The && and || operators use short-circuit evaluation: they stop evaluating as soon as the result is determined. This is both an optimization and a safety feature -- you can safely check for null before accessing a property using &&.

```
bool a = true, b = false;
print(a && b);   // false  -- AND: true only if BOTH are true
print(a || b);   // true   -- OR:  true if AT LEAST ONE is true
print(!a);       // false  -- NOT: flips the boolean

// Short-circuit evaluation -- right side not evaluated if unnecessary:
int? val = null;
bool safe = val != null && val > 0;  // val>0 skipped -- val is null

String? name = null;
bool ok = name != null && name.isNotEmpty;  // name.isNotEmpty not called

// Ternary operator -- compact if/else expression:
int x = 10;
String label = x > 5 ? 'big' : 'small';  // 'big'

// Cascade operator (..) -- call multiple methods on the same object:
// Instead of: lst.add(1); lst.add(2); lst.add(3);
List<int> nums = []
  ..add(1)   // returns the List, not void
  ..add(2)
  ..add(3);  // nums is [1, 2, 3]

// Spread (...) -- unpack an iterable into another collection:
List<int> extra = [4, 5, 6];
List<int> all   = [1, 2, 3, ...extra];  // [1, 2, 3, 4, 5, 6]
```

3.4 Type Test Operators

Dart's type test operators let you check or cast types at runtime. The 'is' operator checks if an object is of a given type and PROMOTES the variable inside the if block (Dart knows the type without a cast). The 'as' operator casts but throws if the type is wrong.

```
// is -- check if object is of a type (returns bool):
dynamic val = 'Hello';
print(val is String);   // true
print(val is int);      // false

// is! -- check if object is NOT of a type:
print(val is! int);     // true

// Type PROMOTION -- after 'is' check, Dart upgrades the type:
Object obj = 'hello world';
if (obj is String) {
  // Inside this block, obj is treated as String -- no cast needed!
  print(obj.toUpperCase());   // 'HELLO WORLD'
  print(obj.split(' '));      // ['hello', 'world']
}

// as -- explicit type cast (throws TypeError if wrong type):
dynamic data = 'Dart';
String s = data as String;   // Safe: data IS a String
// int n = data as int;      // Throws! data is String, not int

// Safe cast pattern -- check before casting:
if (data is int) {
  int n = data;   // Promoted -- no 'as' needed
}
```

3.5 Assignment & Bitwise Operators

Compound assignment operators are shortcuts that modify a variable and reassign it. Bitwise operators work on the binary representation of integers and are useful for flags, masks, and low-level operations.

```
// Compound assignment -- shorthand for x = x op y:
int x = 10;
x += 5;    // x = x + 5 = 15
x -= 3;    // x = x - 3 = 12
x *= 2;    // x = x * 2 = 24
x ~/= 5;   // x = x ~/ 5 = 4
x %= 3;    // x = x % 3 = 1
```

```
// Null-aware assignment -- assign only if null:
String? name;
name ??= 'Default';   // name is null, so assigned 'Default'
name ??= 'Other';     // name is not null now, so skipped

// Bitwise operators -- work on binary representation:
// Each bit position is an independent flag
print(0b1010 & 0b1100);  // 0b1000 = 8  -- AND: bits set in BOTH
print(0b1010 | 0b1100);  // 0b1110 = 14 -- OR: bits set in EITHER
print(0b1010 ^ 0b1100);  // 0b0110 = 6  -- XOR: bits different
print(~0b1010);           // bitwise NOT -- flips all bits
print(1 << 3);            // 8  -- left shift: multiply by 2^3
print(16 >> 2);           // 4  -- right shift: divide by 2^2
```

Chapter 4 -- Strings

Strings in Dart are immutable sequences of UTF-16 code units. 'Immutable' means that every string operation -- like toUpperCase() or replace() -- creates a new string rather than modifying the original. Strings are objects with a rich set of built-in methods for searching, transforming, and splitting.

4.1 String Creation

Dart offers several string literal forms. Single and double quotes are interchangeable. Triple quotes (''' or """) span multiple lines. Raw strings (prefix r) treat backslashes as literal characters, which is essential for regular expressions and Windows file paths.

```
// Single and double quotes are equivalent:
String s1 = 'Single quotes work fine';
String s2 = "Double quotes work too";
String s3 = 'It\'s easy to escape apostrophes';
String s4 = "Or use double quotes for it's convenience";

// Multi-line string -- triple quotes preserve newlines:
String multiLine = '''
First line
Second line
Third line''';

// Raw string -- backslashes treated as literal characters:
String winPath = r'C:\Users\Alice\Documents';  // not an escape
String regex   = r'\d+\.\d+';  // perfect for regex patterns

// Adjacent string literals auto-concatenate (no + needed):
String msg = 'This is a very long message '
    'that spans multiple source lines '
    'but is one string at runtime.';
```

4.2 String Properties

Strings have several useful properties. length counts UTF-16 code units (not necessarily visible characters for emojis). isEmpty and isNotEmpty are more expressive than checking length == 0.

```
String s = 'Hello, Dart!';

print(s.length);       // 13 -- number of UTF-16 code units
print(s.isEmpty);      // false -- same as s.length == 0
print(s.isNotEmpty);   // true  -- same as s.length > 0
print(s[0]);           // 'H'  -- first character (0-indexed)
```

```
print(s[s.length-1]);  // '!'  -- last character

// String concatenation -- creates a new string:
String a = 'Hello' + ' ' + 'World';   // 'Hello World'
// Better: use interpolation for readability
String b = '$a, welcome!';            // 'Hello World, welcome!'
```

4.3 String Methods

All string methods return a new string -- they never modify the original. This is because strings are immutable. The most commonly used methods are listed here with examples.

Method / Property	Example	Description & Return Value
`s.toUpperCase()`	`s.toUpperCase()`	Returns new string with ALL letters capitalized
`s.toLowerCase()`	`s.toLowerCase()`	Returns new string with all letters lowercased
`s.trim()`	`s.trim()`	Remove leading and trailing whitespace. Returns new string
`s.trimLeft()`	`s.trimLeft()`	Remove only leading (left) whitespace
`s.trimRight()`	`s.trimRight()`	Remove only trailing (right) whitespace
`s.contains(p)`	`s.contains('art')`	True if string contains pattern (String or RegExp)
`s.startsWith(p)`	`s.startsWith('He')`	True if string begins with pattern
`s.endsWith(p)`	`s.endsWith('!')`	True if string ends with pattern
`s.indexOf(p)`	`s.indexOf('o')`	Index of first occurrence. Returns -1 if not found
`s.lastIndexOf(p)`	`s.lastIndexOf('l')`	Index of last occurrence. Returns -1 if not found
`s.substring(s,e)`	`s.substring(0, 5)`	Extract characters from start to end (exclusive)
`s.split(pattern)`	`s.split(',')`	Split into List<String> at every delimiter
`s.replaceAll(p,r)`	`s.replaceAll('l','L')`	Replace ALL occurrences of pattern with replacement
`s.replaceFirst(p,r)`	`s.replaceFirst('l','L')`	Replace FIRST occurrence only
`s.padLeft(w,c)`	`s.padLeft(5,'0')`	Pad on left to minimum width w with character c
`s.padRight(w,c)`	`s.padRight(5,'-')`	Pad on right to minimum width w with character c
`s.codeUnitAt(i)`	`s.codeUnitAt(0)`	UTF-16 code unit (number) at index i
`s.compareTo(other)`	`s.compareTo('b')`	Lexicographic compare: negative/0/positive
`s.replaceAllMapped(p,f)`	`s.replaceAllMapped(re,fn)`	Replace matches using a function for each match

```
String s = '  Hello, Dart!  ';
```

```
// Case conversion:
print(s.toUpperCase());        // '  HELLO, DART!  '
print(s.trim().toLowerCase()); // 'hello, dart!'

// Search:
print(s.contains('Dart'));     // true
print(s.indexOf('Dart'));      // 9 (index in original string)
print(s.startsWith('  He'));   // true

// Extract & split:
print(s.trim().substring(0, 5));  // 'Hello'
print('a,b,c'.split(','));        // ['a', 'b', 'c']

// Replace:
print(s.replaceAll('l', 'L'));  // '  HeLLo, Dart!  '

// Padding -- useful for formatting numbers:
print('7'.padLeft(3, '0'));     // '007'
print('hi'.padRight(6, '-'));   // 'hi----'
```

Chapter 5 -- Control Flow & Loops

Control flow determines the order in which your code executes. Dart provides the standard set: if/else for decisions, for/while loops for repetition, and switch for multi-branch selection. Dart 3 adds powerful switch expressions and pattern matching. Exception handling with try/catch ensures your program handles errors gracefully.

5.1 if / else if / else

The if statement evaluates a boolean condition. Only the first matching branch executes. Unlike JavaScript, Dart does NOT have truthy/falsy values -- the condition must be exactly true or false (boolean).

```
int score = 85;

// Standard if-else chain:
if (score >= 90) {
  print('Grade: A');
} else if (score >= 80) {
  print('Grade: B');   // This branch runs
} else if (score >= 70) {
  print('Grade: C');
} else {
  print('Grade: F');   // Fallback
}

// Ternary operator -- compact single-expression if/else:
String result = score >= 60 ? 'Pass' : 'Fail';   // 'Pass'

// IMPORTANT: Dart has NO truthy/falsy -- must be boolean:
// if (score) { }     // COMPILE ERROR! int is not bool
// if (name) { }      // COMPILE ERROR! String is not bool
if (score > 0) { }    // Correct -- explicit comparison
if (name.isNotEmpty) { } // Correct -- explicitly boolean
```

5.2 switch Statement & Expression

The switch statement compares a value against multiple cases. Dart 3 introduced switch EXPRESSIONS (lowercase switch as a value) which are exhaustive -- the compiler verifies all cases are handled. Switch expressions can also use patterns and guard clauses.

```
// Classic switch statement:
String day = 'Monday';
switch (day) {
  case 'Saturday':
  case 'Sunday':            // Multiple cases, same body
    print('Weekend!');
```

```
    break;                  // break is required in switch statements
  case 'Monday':
    print('Start of week');
    break;
  default:                  // Runs if no case matches
    print('Other weekday');
}

// Switch EXPRESSION (Dart 3) -- returns a value, exhaustive:
String type = switch (day) {
  'Saturday' || 'Sunday' => 'Weekend',  // OR pattern
  'Monday'               => 'Start of week',
  String d when d.endsWith('day') => 'Weekday: $d',  // guard
  _                      => 'Unknown', // _ is wildcard/default
};

// Switch with numeric patterns:
int n = 42;
String size = switch (n) {
  < 0   => 'negative',
  0     => 'zero',
  1..10 => 'small',   // range pattern (Dart 3)
  _     => 'large',
};
```

5.3 for Loops

Dart has three forms of for loops, each suited for different situations. The standard for loop gives you an index counter. The for-in loop iterates over any Iterable. forEach() is a higher-order function that accepts a callback.

```
// Standard for loop -- use when you need the index:
for (int i = 0; i < 5; i++) {
  print('Index: $i');   // 0, 1, 2, 3, 4
}

// for-in loop -- iterate over any Iterable (List, Set, etc.):
List<String> fruits = ['apple', 'banana', 'cherry'];
for (String fruit in fruits) {
  print(fruit);   // apple, banana, cherry
}

// forEach() -- higher-order function with a callback:
fruits.forEach((f) => print(f));

// Get both index and value using asMap():
fruits.asMap().forEach((index, value) {
  print('$index: $value');   // 0: apple, 1: banana, etc.
```

```
});

// Functional alternatives -- often cleaner:
var upper = fruits.map((f) => f.toUpperCase()).toList();
// ['APPLE', 'BANANA', 'CHERRY']
```

5.4 while & do-while Loops

Use while when you want to check the condition BEFORE the first iteration (the loop might run 0 times). Use do-while when the body must run at LEAST once before checking the condition.

```
// while -- condition checked BEFORE each iteration:
int n = 1;
while (n < 100) {
  n *= 2;   // doubles n each time
}
print(n);   // 128 (first power of 2 >= 100)

// do-while -- body runs at least ONCE, then condition is checked:
int attempts = 0;
do {
  attempts++;
  print('Attempt $attempts');
} while (attempts < 3);
// Prints: Attempt 1, Attempt 2, Attempt 3

// Common while pattern -- read until a condition is met:
String? input;
while (input != 'quit') {
  input = readLine();   // Gets user input
  process(input);
}
```

5.5 break, continue & Labels

'break' immediately exits the innermost loop. 'continue' skips the rest of the current iteration and jumps to the next one. Labels let you break or continue an OUTER loop from inside a nested loop -- this avoids needing a flag variable.

```
// break -- exit the loop entirely:
for (int i = 0; i < 10; i++) {
  if (i == 5) break;   // Stop when i reaches 5
  print(i);             // 0, 1, 2, 3, 4
}

// continue -- skip this iteration, go to next:
for (int i = 0; i < 10; i++) {
```

```
  if (i % 2 == 0) continue;  // Skip even numbers
  print(i);   // 1, 3, 5, 7, 9
}

// Labels -- named markers for break/continue on outer loops:
outer: for (int i = 0; i < 3; i++) {
  for (int j = 0; j < 3; j++) {
    if (j == 1) continue outer;  // Skip to next i (skip inner loop)
    if (i == 2) break outer;     // Exit BOTH loops
    print('($i, $j)');
  }
}
// Output: (0, 0) then (1, 0) -- labels give you fine-grained control
```

5.6 Exception Handling

Exceptions represent error conditions your program should handle gracefully. 'try' wraps code that might fail. 'on TypeName' catches a specific exception type. 'catch' captures the exception object. 'finally' always runs -- whether or not an exception occurred -- perfect for cleanup code like closing files.

```
// Basic try-catch-finally structure:
try {
  int result = 10 ~/ 0;   // throws IntegerDivisionByZeroException
} on IntegerDivisionByZeroException {
  print('Cannot divide by zero');
} on FormatException catch (e) {
  print('Format error: $e');  // e is the exception object
} catch (e, stackTrace) {
  print('Unknown error: $e');
  print(stackTrace);  // full call stack at point of error
} finally {
  print('Always runs -- good for cleanup (closing files, etc.)');
}

// Throw an exception (any object can be thrown):
throw FormatException('Invalid input format');
throw ArgumentError.value(age, 'age', 'Must be non-negative');
throw Exception('Something went wrong');

// Custom exception class:
class ValidationException implements Exception {
  final String message;
  ValidationException(this.message);
  @override String toString() => 'Validation failed: $message';
}

throw ValidationException('Email is required');
```

Chapter 6 -- Functions

Functions are first-class objects in Dart -- you can store them in variables, pass them as arguments, and return them from other functions. Dart supports named parameters, optional parameters, default values, closures, and generator functions. Mastering Dart functions is essential for writing clean, reusable code.

6.1 Function Basics & Arrow Syntax

A function declaration specifies the return type, name, parameters, and body. When a function body is a single expression, the arrow syntax (=>) is a concise shorthand. Functions with no return value use 'void'. Dart automatically returns null from void functions.

```
// Regular function -- explicit return type and body:
int add(int a, int b) {
  return a + b;
}

// Arrow syntax -- single expression, implicit return:
int multiply(int a, int b) => a * b;
bool isEven(int n) => n % 2 == 0;

// void function -- no return value:
void greet(String name) {
  print('Hello, $name!');
}

// Functions are first-class -- store in a variable:
Function myAdd = add;
print(myAdd(3, 4));   // 7 -- call through the variable

// Pass functions as arguments:
List<int> nums = [1, 2, 3, 4, 5];
nums.where(isEven).forEach(print);  // 2, 4
```

6.2 Optional & Named Parameters

Named parameters (in curly braces {}) must be referenced by name at the call site, making code self-documenting. Positional optional parameters (in square brackets []) are called by position. 'required' marks a named parameter as mandatory. Default values are used when a parameter is omitted.

```
// Named parameters -- called by name, order doesn't matter:
void createUser({required String name, int age = 18, String? email}) {
  print('$name, age $age, email: ${email ?? 'none'}');
}
createUser(name: 'Alice');                    // Alice, age 18, none
```

```
createUser(name: 'Bob', age: 25, email: 'b@example.com');
// createUser(age: 25);   // COMPILE ERROR! name is required

// Positional optional parameters -- in square brackets:
String greet(String name, [String prefix = 'Hello', String? suffix]) {
  return '$prefix, $name${suffix != null ? ', $suffix' : ''}!';
}
print(greet('Alice'));              // Hello, Alice!
print(greet('Bob', 'Hi'));          // Hi, Bob!
print(greet('Carol', 'Hey', 'PhD'));// Hey, Carol, PhD!

// Mix positional and named (positional first):
void log(String message, {String level = 'INFO'}) {
  print('[$level] $message');
}
log('Server started');              // [INFO] Server started
log('Disk full', level: 'ERROR');   // [ERROR] Disk full
```

6.3 Anonymous Functions & Closures

Anonymous functions (lambdas) are functions without a name -- they are defined inline and passed directly. Closures are anonymous functions that 'close over' (capture and remember) variables from their surrounding scope, even after that scope has finished executing.

```
// Anonymous function stored in a variable:
var square = (int x) => x * x;
print(square(5));   // 25

// Passed directly to higher-order functions:
List<int> nums = [1, 2, 3, 4, 5];
List<int> squared = nums.map((x) => x * x).toList();  // [1,4,9,16,25]
List<int> evens   = nums.where((x) => x.isEven).toList(); // [2,4]
int total = nums.reduce((acc, x) => acc + x);   // 15

// Closure -- captures surrounding variable 'n':
Function makeAdder(int n) {
  // The inner function 'closes over' n:
  return (int x) => x + n;   // n is remembered even after makeAdder returns
}

var add5  = makeAdder(5);   // n=5 is captured
var add10 = makeAdder(10);  // n=10 is captured separately
print(add5(3));   // 8    (3 + 5)
print(add10(3));  // 13   (3 + 10)
print(add5(7));   // 12   (7 + 5) -- n=5 is still remembered
```

6.4 Generators (sync* & async*)

Generators produce sequences of values lazily -- they only compute each value when requested. sync* generators return an Iterable and use 'yield' to produce values one at a time. This is memory-efficient for large or infinite sequences because values are not all computed upfront.

```
// sync* -- produces Iterable<T> lazily:
Iterable<int> range(int start, int end) sync* {
  // yield pauses, returns value, then resumes on next request:
  for (int i = start; i < end; i++) {
    yield i;   // produces one value at a time
  }
}

for (int x in range(1, 6)) {
  print(x);   // 1, 2, 3, 4, 5 -- computed one at a time
}

// yield* -- delegate to another Iterable (flattens one level):
Iterable<int> combined(Iterable<int> a, Iterable<int> b) sync* {
  yield* a;   // yield all from a
  yield* b;   // then all from b
}

// Infinite sequence -- only safe with lazy evaluation!
Iterable<int> naturals() sync* {
  int n = 0;
  while (true) yield n++;  // infinite, but only computed when needed
}

// Take only first 5 -- naturals() is NOT evaluated fully:
print(naturals().take(5).toList());  // [0, 1, 2, 3, 4]

// async* -- produces Stream<T> (async sequence):
Stream<int> tickStream() async* {
  for (int i = 1; i <= 5; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i;   // emits one value per second
  }
}
```

Chapter 7 -- OOP: Classes & Objects

Dart is a fully object-oriented language -- everything is an object, including numbers and functions. Classes are blueprints for creating objects. They bundle data (fields) and behavior (methods) together. Dart supports constructors, inheritance, abstract classes, interfaces (via 'implements'), and mixins for code reuse.

7.1 Classes & Constructors

A class defines fields (instance variables) and methods. Constructors initialize objects. Dart's shorthand 'this.fieldName' in constructor parameters automatically assigns the argument to the field of the same name -- a very common pattern. Named constructors create additional ways to instantiate objects (e.g., from JSON).

```
class Person {
  // Instance fields -- each object has its own copy:
  String name;
  int age;

  // Default constructor -- 'this.name' assigns argument to field:
  Person(this.name, this.age);

  // Named constructor -- alternative way to create objects:
  Person.anonymous() : name = 'Anonymous', age = 0;

  // Factory constructor -- returns existing or computed instance:
  factory Person.fromJson(Map<String, dynamic> json) {
    return Person(json['name'] as String, json['age'] as int);
  }

  // Instance method:
  String greet() => 'Hi! I am $name, $age years old.';

  // Override toString for readable output:
  @override
  String toString() => 'Person($name, $age)';
}

// Creating objects:
var p1 = Person('Alice', 30);
var p2 = Person.anonymous();           // Uses named constructor
var p3 = Person.fromJson({'name': 'Bob', 'age': 25});

print(p1.greet());   // Hi! I am Alice, 30 years old.
print(p3);           // Person(Bob, 25) -- uses toString()
```

7.2 Getters, Setters & Static Members

Getters and setters are special methods that look like property access but run code. Getters compute values on the fly. Setters validate data before assignment. Static members belong to the class itself, not individual instances -- they are shared across all objects of that class.

```
class Rectangle {
  double width, height;
  Rectangle(this.width, this.height);

  // Getter -- accessed like a property (no parentheses):
  double get area      => width * height;
  double get perimeter => 2 * (width + height);
  bool   get isSquare  => width == height;

  // Setter -- validates before assigning:
  set width(double value) {
    if (value <= 0) throw ArgumentError('Width must be positive');
    width = value;   // sets the backing field
  }

  // Static members -- belong to the class, not instances:
  static const String shapeName = 'Rectangle';
  static Rectangle square(double side) => Rectangle(side, side);
}

var rect = Rectangle(4, 5);
print(rect.area);          // 20.0 -- getter called like a property
print(rect.perimeter);     // 18.0
print(rect.isSquare);      // false
print(Rectangle.shapeName);// 'Rectangle' -- static access via class name
var sq = Rectangle.square(3); // 3x3 square via static factory
```

7.3 Inheritance & Polymorphism

Inheritance ('extends') lets a child class reuse and extend a parent class. The child inherits all fields and methods. 'super' refers to the parent class. '@override' marks a method that replaces a parent's version. Polymorphism means you can treat objects of different subclasses uniformly through the parent type.

```
class Animal {
  String name;
  Animal(this.name);
  void speak() => print('...');  // default implementation
  @override String toString() => 'Animal($name)';
}

class Dog extends Animal {
```

```
  String breed;
  // super(name) calls the parent constructor:
  Dog(String name, this.breed) : super(name);

  @override  // replaces Animal's speak():
  void speak() => print('$name barks: Woof!');
}

class Cat extends Animal {
  Cat(String name) : super(name);
  @override
  void speak() => print('$name meows: Meow!');
}

// Polymorphism -- treat all as Animal, call correct version:
List<Animal> animals = [Dog('Rex', 'Labrador'), Cat('Whiskers')];
for (var a in animals) {
  a.speak();  // Calls Dog.speak() or Cat.speak() dynamically
}
// Rex barks: Woof!
// Whiskers meows: Meow!
```

7.4 Abstract Classes, Interfaces & Mixins

Abstract classes define a contract without full implementation -- subclasses MUST implement abstract members. In Dart, every class defines an interface implicitly -- 'implements' means you commit to providing all the methods without inheriting the code. Mixins are reusable bundles of code that can be added to any class using 'with'.

```
// Abstract class -- cannot be instantiated directly:
abstract class Shape {
  // Abstract getter -- no implementation, subclass MUST provide:
  double get area;
  // Concrete method -- shared by all subclasses:
  void describe() => print('${runtimeType} with area $area');
}

class Circle extends Shape {
  double radius;
  Circle(this.radius);
  @override double get area => 3.14159 * radius * radius;
}

// Mixin -- a bag of methods added with 'with':
mixin Flyable {
  void fly() => print('${runtimeType} is flying!');
  int get altitude => 1000;
}
```

```
mixin Swimmable {
  void swim() => print('${runtimeType} is swimming!');
}

// A class can use multiple mixins simultaneously:
class Duck extends Animal with Flyable, Swimmable {
  Duck(String name) : super(name);
}

var duck = Duck('Donald');
duck.fly();   // Duck is flying!
duck.swim();  // Duck is swimming!
duck.speak(); // ...  (from Animal)
```

Chapter 8 -- Async Programming

Dart is single-threaded and uses an event loop for concurrency. 'Async programming' means starting long operations (network calls, file I/O) and continuing with other work while waiting. Future represents a value that will be available later. async/await is syntactic sugar that makes async code look and read like synchronous code -- dramatically reducing complexity compared to callback-based approaches.

8.1 Future & async/await Basics

A Future<T> is a promise that T will be available at some point. 'async' marks a function as asynchronous (it will return a Future). 'await' pauses execution of the current async function until the Future completes -- but crucially, it does NOT block the entire program. Other events can still be processed while waiting.

```
// Simulates a network request that takes 2 seconds:
Future<String> fetchUserName(int id) async {
  await Future.delayed(Duration(seconds: 2));  // non-blocking wait
  return 'User_$id';  // returned value wraps in Future automatically
}

// await can only be used inside an async function:
Future<void> main() async {
  print('Fetching user...');
  String name = await fetchUserName(42);   // waits, then unwraps
  print('Got: $name');   // 'Got: User_42' (after 2 seconds)
  print('Done!');
}

// Callback style (older approach -- harder to read):
fetchUserName(42)
  .then((name) => print('Name: $name'))
  .catchError((e) => print('Error: $e'))
  .whenComplete(() => print('Always runs'));

// Run multiple Futures in PARALLEL (not sequential):
var results = await Future.wait([
  fetchUserName(1),   // All three start simultaneously
  fetchUserName(2),
  fetchUserName(3),
]);
// Total time = max of the three, not sum
print(results);   // ['User_1', 'User_2', 'User_3']
```

8.2 Error Handling in Async Code

Exceptions thrown inside async functions propagate through the Future chain. You can catch them with try/catch (when using await) or with .catchError() (callback style). Always handle errors in async code -- an unhandled Future error can crash your app or cause silent failures.

```
// Async function that might fail:
Future<int> parseRemoteNumber(String url) async {
  String response = await fetchFromUrl(url);  // might throw
  return int.parse(response);  // might throw FormatException
}

// try/catch works perfectly with async/await:
Future<void> main() async {
  try {
    int number = await parseRemoteNumber('http://api.example.com/n');
    print('Number: $number');
  } on FormatException catch (e) {
    print('Invalid format: $e');
  } catch (e) {
    print('Network or other error: $e');
  } finally {
    print('Cleanup -- always runs');
  }
}

// Safe operation with default value on failure:
Future<int> safeParseNumber(String url) async {
  try {
    return await parseRemoteNumber(url);
  } catch (_) {
    return 0;   // default value if anything goes wrong
  }
}

// Timeout -- fail if takes too long:
var result = await fetchData()
  .timeout(Duration(seconds: 5));  // throws TimeoutException
```

8.3 Streams -- Async Sequences

A Stream is like a Future but for multiple values over time -- think of it as an asynchronous Iterable. Streams are used for UI events, WebSocket data, file reading, and any data that arrives in pieces. 'await for' is the cleanest way to process stream values sequentially.

```
// Create a Stream with async*:
Stream<int> countDown(int from) async* {
  for (int i = from; i >= 0; i--) {
    await Future.delayed(Duration(seconds: 1));
```

```
    yield i;   // emits one value per second
  }
}

// await for -- process each value as it arrives:
await for (int n in countDown(5)) {
  print(n);   // 5, 4, 3, 2, 1, 0 (one per second)
}

// .listen() -- callback-based (useful for broadcast streams):
countDown(3).listen(
  (data) => print('Got: $data'),
  onError: (e) => print('Error: $e'),
  onDone: () => print('Stream complete'),
);

// Transform streams with functional operations:
countDown(10)
  .where((n) => n.isEven)    // only even numbers
  .map((n) => 'Even: $n')   // transform to strings
  .listen(print);
// 'Even: 10', 'Even: 8', 'Even: 6', 'Even: 4', 'Even: 2', 'Even: 0'
```

Chapter 9 -- Generics & Iterables

Generics let you write code that works with any type while preserving type safety. Instead of writing separate functions for int, String, and double, you write one generic function with a type parameter T. The caller specifies T, and the compiler checks everything matches. Dart's entire collection library (List, Set, Map) is built on generics.

9.1 Generic Classes

A generic class uses a type parameter T as a placeholder. When you create an instance, you provide the actual type. The compiler then enforces type correctness for all operations on that instance -- you cannot accidentally put an int into a Box<String>.

```
// Generic class -- T is a placeholder for any type:
class Box<T> {
  T value;
  Box(this.value);
  T get item => value;
  bool get isEmpty => false; // could check for sentinel
  @override String toString() => 'Box<$T>($value)';
}

// T is replaced with specific types at usage:
var intBox = Box<int>(42);     // T = int
var strBox = Box<String>('hi');// T = String
print(intBox.item + 1);        // 43 -- type-safe: item is int
print(strBox.item.toUpperCase()); // 'HI' -- item is String

// Bounded generic -- restrict T to types that are Comparable:
class MinMaxFinder<T extends Comparable<T>> {
  T findMin(List<T> lst) => lst.reduce((a,b) => a.compareTo(b)<0?a:b);
  T findMax(List<T> lst) => lst.reduce((a,b) => a.compareTo(b)>0?a:b);
}

var finder = MinMaxFinder<int>();
print(finder.findMin([3,1,5,2]));   // 1
print(finder.findMax([3,1,5,2]));   // 5
// Also works for strings, doubles, etc.
```

9.2 Generic Functions

Generic functions have type parameters attached to the function itself. Dart usually infers the type from the arguments so you rarely need to write it explicitly. Multiple type parameters are possible for functions that operate on pairs of types.

```
// Generic function -- T inferred from argument:
```

```
T first<T>(List<T> lst) => lst.first;
T? firstOrNull<T>(List<T> lst) => lst.isEmpty ? null : lst.first;

print(first<int>([1, 2, 3]));    // 1 -- explicit
print(first(['a', 'b']));         // 'a' -- inferred T=String

// Multiple type parameters:
Map<V, K> invertMap<K, V>(Map<K, V> map) {
  return {for (var e in map.entries) e.value: e.key};
}

var original = {'a': 1, 'b': 2};
var inverted = invertMap(original);   // {1:'a', 2:'b'}

// Generic with transform:
List<R> mapList<T, R>(List<T> lst, R Function(T) fn) =>
  lst.map(fn).toList();

print(mapList([1,2,3], (n) => '$n'));  // ['1','2','3']
print(mapList(['a','bb','ccc'], (s) => s.length));  // [1,2,3]
```

9.3 All Iterable Methods

Every Dart collection (List, Set, Iterable) shares the Iterable<E> interface. These methods provide powerful functional-style operations for transforming, filtering, and aggregating data. Many are lazy -- they only compute values when requested, saving memory.

Method / Property	Example	Description & Return Value
`it.map(f)`	`nums.map((x)=>x*2)`	Transform each element. Returns LAZY Iterable (call .toList() to evaluate)
`it.where(test)`	`nums.where((x)=>x>3)`	Filter elements keeping those where test returns true. Returns lazy Iterable
`it.expand(f)`	`lst.expand((x)=>[x,x])`	FlatMap: transform each element to Iterable, then flatten one level
`it.reduce(f)`	`nums.reduce((a,b)=>a+b)`	Combine all elements into one value. Throws StateError if empty
`it.fold(init,f)`	`nums.fold(0,(a,b)=>a+b)`	Like reduce but with an initial value. Safe on empty iterables
`it.any(test)`	`nums.any((x)=>x>4)`	True if AT LEAST ONE element passes test. Short-circuits on first true
`it.every(test)`	`nums.every((x)=>x>0)`	True if ALL elements pass test. Short-circuits on first false
`it.contains(val)`	`nums.contains(3)`	True if value exists. O(n) for most iterables
`it.first`	`nums.first`	First element. Throws StateError if iterable is empty
`it.last`	`nums.last`	Last element. Throws StateError if iterable is empty

Method / Property	Example	Description & Return Value
`it.single`	`nums.single`	Only element. Throws if iterable has 0 or more than 1 elements
`it.firstWhere(t)`	`nums.firstWhere((x)=>x>3)`	First element passing test. Use orElse: ()=>default for safety
`it.lastWhere(t)`	`nums.lastWhere((x)=>x>3)`	Last element passing test
`it.take(n)`	`nums.take(3)`	First n elements as lazy Iterable. Safe if iterable is shorter
`it.skip(n)`	`nums.skip(2)`	Skip first n elements, return rest as lazy Iterable
`it.takeWhile(t)`	`nums.takeWhile((x)=>x<5)`	Take elements while test is true, stop at first false
`it.skipWhile(t)`	`nums.skipWhile((x)=>x<3)`	Skip elements while test is true, yield rest
`it.toList()`	`nums.toList()`	Materialize all elements into a new growable List
`it.toSet()`	`nums.toSet()`	Materialize into a new Set (removes duplicates)
`it.join(sep)`	`nums.join(', ')`	Concatenate all elements into a String with separator
`it.length`	`nums.length`	Total count of elements. May iterate the whole iterable
`it.isEmpty`	`nums.isEmpty`	True if no elements. Preferred over length==0 (may be faster)
`it.isNotEmpty`	`nums.isNotEmpty`	True if at least one element exists
`it.whereType<T>()`	`lst.whereType<String>()`	Filter and cast to type T in one step
`it.followedBy(it2)`	`a.followedBy(b)`	Lazily concatenate two iterables without creating a new list
`it.elementAt(i)`	`nums.elementAt(2)`	Element at index i. O(n) for most types except List

```
List<int> nums = [1, 2, 3, 4, 5];

// Chaining operations -- each is lazy, only evaluated at .toList():
var result = nums
  .where((x) => x.isOdd)          // [1, 3, 5]
  .map((x) => x * x)              // [1, 9, 25]
  .where((x) => x > 5)            // [9, 25]
  .toList();                       // evaluate now
print(result);   // [9, 25]

// Aggregate:
print(nums.fold(0, (sum, x) => sum + x));       // 15 -- sum
print(nums.fold(1, (prod, x) => prod * x));     // 120 -- product

// Checks:
print(nums.any((x) => x > 10));     // false
print(nums.every((x) => x >= 1));   // true

// Type filtering:
List<Object> mixed = [1, 'hello', 2, 'world', 3];
```

```
var strings = mixed.whereType<String>().toList(); // ['hello','world']
var numbers = mixed.whereType<int>().toList();    // [1, 2, 3]
```

Chapter 10 -- Enums

An enum (enumeration) defines a finite set of named constant values. They are perfect when a variable should only ever be one of a fixed set of options -- like days of the week, HTTP methods, or order statuses. Dart's enhanced enums (since 2.17) can have fields, methods, and implement interfaces, making them far more powerful than enums in most languages.

10.1 Simple Enums

Simple enums just define a list of named constants. Each value has a built-in .name (its string name), .index (its position, starting at 0), and you can iterate all values with .values. The .byName() method lets you parse a string back to an enum value.

```
// Define an enum -- a fixed set of named constants:
enum Direction { north, south, east, west }
enum Status    { pending, active, paused, stopped }
enum Color     { red, green, blue, yellow }

// Using enum values:
Direction current = Direction.north;
print(current);           // Direction.north -- full name
print(current.name);      // 'north'  -- just the value name
print(current.index);     // 0        -- 0-based position

// List all values:
print(Direction.values); // [north, south, east, west]

// Comparison:
print(current == Direction.north);  // true
print(current == Direction.south);  // false

// Parse from string -- useful for JSON/API data:
Status s = Status.values.byName('active');  // Status.active

// Use in switch (exhaustive in Dart 3 -- no default needed!):
String label = switch (current) {
  Direction.north => 'N',
  Direction.south => 'S',
  Direction.east  => 'E',
  Direction.west  => 'W',
};   // Compiler verifies all 4 cases are covered
```

10.2 Enhanced Enums (Dart 2.17+)

Enhanced enums can carry data alongside each constant. Add fields and a const constructor, then each enum value provides its field arguments. You can also add methods, getters, and implement interfaces -- the enum values become full objects with behavior.

```
// Enhanced enum -- each value carries data:
enum Planet {
  // Each value calls the const constructor with gravity:
  mercury(3.7),
  venus(8.87),
  earth(9.81),   // <-- trailing semicolon after last value
  mars(3.72);

  // Const field -- immutable data for each enum value:
  final double gravity;

  // Const constructor -- required for enhanced enums:
  const Planet(this.gravity);

  // Method -- each enum value can call this:
  double weightOn(double mass) => mass * gravity;

  // Getter -- computed property:
  bool get isHabitable => this == Planet.earth;
}

print(Planet.earth.gravity);           // 9.81
print(Planet.mars.weightOn(70));       // 260.4
print(Planet.earth.isHabitable);       // true
print(Planet.mercury.name);            // 'mercury'

// Iterate all values and call methods:
for (var p in Planet.values) {
  print('${p.name}: ${p.gravity}m/s^2');
}
```

10.3 Enum Implementing an Interface

Enums can implement interfaces and even use mixins, making them true first-class objects. This is useful when different enum values need to provide different implementations of a contract.

```
// Define a mixin or interface:
mixin Describable { String get description; }

// Enum implementing the mixin:
enum HttpMethod with Describable {
  get('Retrieve a resource, no side effects'),
```

```
  post('Create a new resource'),
  put('Replace an existing resource'),
  patch('Partially update a resource'),
  delete('Remove a resource');

  final String _desc;
  const HttpMethod(this._desc);
  @override String get description => _desc;

  // Computed properties:
  bool get isReadOnly => this == HttpMethod.get;
  bool get hasBody => this == post || this == put || this == patch;
}

print(HttpMethod.post.description);  // 'Create a new resource'
print(HttpMethod.get.isReadOnly);    // true
print(HttpMethod.delete.hasBody);    // false
```

Best Practice: Prefer Enums over Constants

Using an enum instead of int or String constants gives you: (1) type safety -- can't pass an invalid value; (2) IDE autocomplete; (3) exhaustive switch checks; (4) .name, .index, .values built in. Always prefer enums for fixed sets.

Chapter 11 -- Extensions

Extensions let you add new methods, getters, setters, and operators to any existing type -- including types from the Dart standard library or third-party packages -- without subclassing or modifying the original source. This is especially powerful for adding utility methods to built-in types like String, int, and List.

11.1 String & Number Extensions

By adding an extension on String, every string in your program gains the new methods. Extensions look like normal method calls at the call site, making your code more readable. The extension can call any method that String already has using 'this'.

```
// Extension on String -- adds methods to ALL strings:
extension StringUtils on String {
  // 'this' refers to the String the method is called on:
  String get reversed => split('').reversed.join();
  bool get isPalindrome => this == reversed;
  String capitalize() =>
    isEmpty ? this : this[0].toUpperCase() + substring(1);
  String truncate(int max, [String suffix = '...']) =>
    length <= max ? this : '${substring(0, max)}$suffix';
  List<String> get words => trim().split(RegExp(r'\s+'));
  bool get isEmail => contains('@') && contains('.');
}

// Use them just like built-in methods:
print('racecar'.isPalindrome);       // true
print('Hello World'.words);          // ['Hello', 'World']
print('Long text here'.truncate(8)); // 'Long tex...'
print('alice@example.com'.isEmail);  // true

// Extension on int:
extension IntUtils on int {
  bool get isPrime {
    if (this < 2) return false;
    for (int i = 2; i <= this ~/ 2; i++)
      if (this % i == 0) return false;
    return true;
  }
  Duration get seconds => Duration(seconds: this);
  List<int> range([int start = 0]) =>
    List.generate(this - start, (i) => i + start);
}

print(7.isPrime);          // true
print(5.range(1));         // [1, 2, 3, 4]
```

```
await Future.delayed(2.seconds);   // much more readable!
```

11.2 Collection Extensions

Generic extensions (on List<T>) work for any element type. This allows writing utility methods like chunked() or distinctBy() that work on any List regardless of what it contains.

```
// Generic extension -- works for List of any type:
extension ListUtils<T> on List<T> {
  // Split list into chunks of size n:
  List<List<T>> chunked(int size) {
    return [for (int i=0; i<length; i+=size)
      sublist(i, (i + size).clamp(0, length))];
  }
  // Null-safe access (no StateError on empty list):
  T? get firstOrNull => isEmpty ? null : first;
  T? get lastOrNull  => isEmpty ? null : last;
  // Remove duplicates while preserving order:
  List<T> get distinct {
    final seen = <T>{};
    return where((e) => seen.add(e)).toList();
  }
}

print([1,2,3,4,5].chunked(2));      // [[1,2],[3,4],[5]]
print(<int>[].firstOrNull);          // null (no exception!)
print([1,2,2,3,1,3].distinct);      // [1, 2, 3]

// Extension on nullable types:
extension NullableX on String? {
  String get orEmpty => this ?? '';
  bool get isNullOrEmpty => this == null || this!.isEmpty;
}

String? name = null;
print(name.orEmpty);         // '' (empty string)
print(name.isNullOrEmpty);   // true
```

Chapter 12 -- Typedefs & Callable Classes

Typedefs create named type aliases. They are most useful for function types -- giving a long function signature a short, descriptive name improves readability and enables reuse. Non-function typedefs (Dart 2.13+) can alias any type, including complex generics. Callable classes are objects that behave like functions.

12.1 Function Typedefs

Without a typedef, you would write 'bool Function(String, int)' everywhere. With a typedef, you write 'Validator' -- once. This makes function-type parameters in APIs much clearer. The typedef can also be generic.

```
// Define named function types:
typedef IntTransformer  = int Function(int);
typedef Predicate<T>    = bool Function(T);
typedef Callback        = void Function(String message);
typedef AsyncFetcher<T> = Future<T> Function(String url);

// Use the typedef -- much cleaner than inline function types:
IntTransformer doubler = (x) => x * 2;
IntTransformer squarer = (x) => x * x;

// Function parameter using typedef -- clear what's expected:
List<int> applyToAll(List<int> list, IntTransformer fn) {
  return list.map(fn).toList();
}

print(applyToAll([1,2,3], doubler));  // [2, 4, 6]
print(applyToAll([1,2,3], squarer));  // [1, 4, 9]

// Generic predicate -- works for any type:
Predicate<int>    isPositive = (n) => n > 0;
Predicate<String> isLong     = (s) => s.length > 5;

print([1,-2,3,-4].where(isPositive).toList()); // [1, 3]
print(['hi','hello world'].where(isLong).toList()); // ['hello world']
```

12.2 Non-function Typedefs & Type Aliases

Since Dart 2.13, typedef can alias ANY type, not just function types. This is especially useful for long generic types like Map<String, dynamic> that appear repeatedly in your codebase.

```
// Alias common complex types for brevity:
typedef JsonMap   = Map<String, dynamic>;  // very common in JSON work
typedef StringMap = Map<String, String>;
```

```
typedef IntList   = List<int>;

// Now your code is much more readable:
JsonMap parseUser(String body) => jsonDecode(body) as JsonMap;
StringMap headers = {'Content-Type': 'application/json'};

// Generic typedef -- parameterized alias:
typedef Pair<A, B> = Map<A, B>;
Pair<String, int> scores = {'Alice': 90, 'Bob': 85};
```

12.3 Callable Classes

A class with a call() method can be invoked using function-call syntax: object(args). This creates objects that ACT like functions but can carry state and configuration between calls. Useful for things like validators, transformers, and middleware that need setup parameters.

```
// Callable class -- has a call() method:
class Multiplier {
  final int factor;
  const Multiplier(this.factor);

  // This makes the object callable like a function:
  int call(int value) => value * factor;
}

var triple = Multiplier(3);
var double2 = Multiplier(2);
print(triple(5));    // 15 -- called exactly like a function!
print(double2(10));  // 20

// More complex: stateful validator object:
class RangeValidator {
  final num min, max;
  const RangeValidator(this.min, this.max);

  bool call(num value) => value >= min && value <= max;
  String errorMsg(num value) =>
    '$value is not in range [$min, $max]';
}

var ageCheck   = RangeValidator(0, 120);
var scoreCheck = RangeValidator(0, 100);

print(ageCheck(25));    // true
print(ageCheck(150));   // false
print(scoreCheck(85));  // true
```

```
// Can be passed as a function:
List<int> valid = [25, 150, -5, 80].where(ageCheck).toList();
// [25, 80]
```

Chapter 13 -- Pattern Matching, Records & Sealed Classes

Dart 3 introduced three landmark features: Records (lightweight value types for bundling multiple values), Patterns (a concise syntax for matching and destructuring data), and Sealed Classes (a way to define closed hierarchies that enable exhaustive switch checking). Together they make Dart's type system much more expressive.

13.1 Records -- Lightweight Value Types

A record is an immutable, anonymous object that bundles multiple values together. Unlike a class, you don't need to define it beforehand. Records are VALUE types -- two records with the same values are equal. They are perfect for returning multiple values from a function without creating a dedicated class.

```
// Positional record -- access values via .$1, .$2, etc.:
(int, String) idAndName = (42, 'Alice');
print(idAndName.$1);   // 42
print(idAndName.$2);   // 'Alice'

// Named record fields -- access via .fieldName:
({String name, int age, bool active}) user =
  (name: 'Bob', age: 25, active: true);
print(user.name);    // 'Bob'
print(user.age);     // 25
print(user.active);  // true

// Records are VALUE types -- equality is by content:
var r1 = (1, 'hello');
var r2 = (1, 'hello');
print(r1 == r2);   // true! (unlike objects, which check identity)

// Return multiple values cleanly -- no class needed:
(int min, int max) findBounds(List<int> lst) {
  lst.sort();
  return (lst.first, lst.last);
}

// Destructuring the record into separate variables:
var (lo, hi) = findBounds([3, 1, 5, 2, 4]);
print('Range: $lo to $hi');   // Range: 1 to 5
```

13.2 Patterns & Destructuring

Patterns provide a concise way to match the structure of data and extract parts of it in one step. They work in variable declarations, switch statements, and for loops. Guards (the 'when' keyword) add conditions to patterns.

```
// List pattern -- destructure a list into named variables:
var [a, b, c] = [1, 2, 3];
print(a);   // 1

// List with rest pattern (...) -- capture the tail:
var [head, ...tail] = [1, 2, 3, 4, 5];
print(head);   // 1
print(tail);   // [2, 3, 4, 5]

// Map pattern -- extract values by key:
var {'name': n, 'age': ag} = {'name': 'Alice', 'age': 30};
print(n);    // 'Alice'
print(ag);   // 30

// Object pattern -- extract fields from a class object:
class Point { int x, y; Point(this.x, this.y); }
// Shorthand ':x' is sugar for 'x: x' when variable name matches field:
var Point(:x, :y) = Point(3, 4);
print('x=$x, y=$y');   // x=3, y=4

// Pattern in switch with type check and guard:
Object val = 42;
switch (val) {
  case int n when n > 100:  // type check + guard condition
    print('Big number: $n');
  case int n when n > 0:
    print('Positive: $n');   // matches
  case int n:
    print('Zero or negative: $n');
  case String s:
    print('String: $s');
}
```

13.3 Switch Expressions (Exhaustive)

Switch expressions return a value directly. In Dart 3, when you switch on a sealed class or enum, the compiler verifies you have covered every case -- if you miss one, it's a compile error. This prevents bugs where you forgot to handle a new enum value added later.

```
// Switch expression -- returns a value, no 'break' needed:
int score = 85;
String grade = switch (score) {
  >= 90 => 'A',     // relational pattern
  >= 80 => 'B',     // first match wins
  >= 70 => 'C',
  >= 60 => 'D',
```

```
  _      => 'F',      // _ is the wildcard (default)
};
print(grade);   // 'B'

// OR pattern -- multiple values for same result:
String day = 'Saturday';
bool isWeekend = switch (day) {
  'Saturday' || 'Sunday' => true,   // either matches
  _ => false,
};
```

13.4 Sealed Classes & Exhaustiveness

A sealed class can only be extended within the same library file. This makes the set of subclasses closed and known to the compiler. When you switch on a sealed class, Dart can verify exhaustiveness -- every subclass must be handled, with no default needed.

```
// sealed -- all subclasses must be in the SAME file:
sealed class Shape {}
class Circle    extends Shape { double r; Circle(this.r); }
class Rectangle extends Shape { double w, h; Rectangle(this.w, this.h); }
class Triangle  extends Shape { double b, h; Triangle(this.b, this.h); }

// Exhaustive switch -- compiler GUARANTEES all cases covered:
double area(Shape s) => switch (s) {
  Circle(:var r)         => 3.14159 * r * r,
  Rectangle(:var w, :var h) => w * h,
  Triangle(:var b, :var h)  => 0.5 * b * h,
  // No default needed -- compiler knows Shape has exactly 3 subtypes
};

print(area(Circle(5)));         // 78.539...
print(area(Rectangle(4, 6)));   // 24.0

// Common sealed Result type -- replaces exceptions for error handling:
sealed class Result<T> {}
class Success<T> extends Result<T> { final T value; Success(this.value); }
class Failure<T> extends Result<T> { final String err; Failure(this.err); }

Result<int> safeDivide(int a, int b) =>
  b == 0 ? Failure('Division by zero') : Success(a ~/ b);

var r = safeDivide(10, 2);
String msg = switch (r) {
  Success(:var value) => 'Result: $value',
  Failure(:var err)   => 'Error: $err',
};
```

```
print(msg);   // 'Result: 5'
```

Chapter 14 -- Advanced OOP: Mixins & Interfaces

Beyond basic inheritance, Dart provides three powerful mechanisms for code reuse and type contracts: mixins (share behavior across unrelated classes), interfaces (define contracts without inheriting code), and the covariant keyword (allow method overrides with more specific types). Understanding when to use each is key to writing well-structured Dart code.

14.1 Mixins with State & on Constraint

Mixins can contain fields, constructors, and state -- not just methods. The 'on' clause restricts which classes can use a mixin. This lets the mixin safely call methods from the required base class without knowing the exact subtype.

```
// Mixin with state -- each class using it gets its own copy of _logs:
mixin Logging {
  final List<String> _logs = [];
  void log(String msg) {
    final entry = '[${DateTime.now()}] $msg';
    _logs.add(entry);
    print(entry);
  }
  List<String> get logs => List.unmodifiable(_logs);
}

// 'on Animal' -- this mixin can ONLY be used on Animal subclasses:
mixin Domestic on Animal {
  String? owner;
  void greetOwner() {
    eat();   // Safe: we know this class has eat() because of 'on Animal'
    print('$name greets ${owner ?? 'nobody'}');
  }
}

// Dog inherits Animal AND gets both mixin behaviors:
class Dog extends Animal with Logging, Domestic {
  Dog(String name) : super(name);
}

var dog = Dog('Rex');
dog.owner = 'Alice';
dog.log('Dog created');
dog.greetOwner();
print(dog.logs.length);   // 1
```

14.2 extends vs implements vs with

These three keywords look similar but do very different things. Choosing the right one makes your design flexible and testable.

Keyword	Inherits code?	Multiple?	Best used when
`extends`	`Yes -- fields, methods, state`	No, one class only	Child IS-A parent. Reuse parent's implementation
`implements`	`No -- must override everything`	Yes, unlimited	Satisfy a type contract. Makes mocking easy in tests
`with`	`Yes -- copies mixin code in`	Yes, unlimited	Cross-cutting behavior: logging, caching, serializing

```
// All three together in one class:
class EnhancedDog extends Animal
    implements Printable, Saveable
    with Logging, Domestic {
  EnhancedDog(String name) : super(name);
  @override void printMe() => log('$name printed');
  @override Future<void> save() async => log('$name saved');
}
```

14.3 Covariant Keyword

Normally, you cannot override a method with a narrower parameter type -- it would break Liskov Substitution Principle. 'covariant' tells Dart to allow it anyway, with a runtime check. This comes up often in frameworks like Flutter where widgets need to refer to their specific state subclass.

```
// Without covariant: Dart rejects the override because Dog
// is a subtype of Animal, which would break type safety generally.

class Animal {
  void eat(Animal food) { print('Eating'); }
}

class Dog extends Animal {
  // covariant: tells Dart this narrowing override is intentional:
  @override
  void eat(covariant Dog food) {  // only accepts Dog, not any Animal
    print('${food.name} eaten');
  }
}

// Dart adds a runtime check -- passing a Cat to Dog.eat() throws

// Most common real-world use -- Flutter StatefulWidget:
abstract class State<T extends StatefulWidget> {
  T get widget;
```

```
  // When overriding, you can narrow T to your specific widget type:
  void didUpdateWidget(covariant T oldWidget) {}
}
```

Chapter 15 -- Constructors (Advanced)

Dart has several types of constructors beyond the basic default constructor. Initializer lists run code before the constructor body -- essential for final fields. Redirecting constructors chain to another constructor. Factory constructors can return cached or subtype instances. Const constructors enable compile-time constants and object canonicalization.

15.1 Initializer Lists

The initializer list (after the colon, before the body) runs before the constructor body and is the only place you can set final fields using computed values. It can also include assert() statements for validation that run in debug mode.

```
class Point {
  final double x, y;  // final: must be set in initializer list

  // Initializer list -- runs BEFORE the constructor body:
  Point(double x, double y)
      : x = x.abs(),         // compute and assign to final field
        y = y.abs(),         // all initializations here
        assert(x >= 0 && y >= 0, 'Point must be in positive quadrant');

  // Redirecting constructor -- calls another constructor via this():
  // No body allowed -- the delegated constructor does all the work.
  Point.origin() : this(0, 0);           // delegates to main ctor
  Point.onXAxis(double x) : this(x, 0);  // creates a point on X axis

  // Const constructor -- enables compile-time constant instances:
  const Point.constOrigin() : x = 0, y = 0;
}

var p1 = Point(-3, -4);       // stored as (3, 4) due to abs()
var p2 = Point.origin();      // (0, 0)
var p3 = Point.onXAxis(5);    // (5, 0)
const p4 = Point.constOrigin(); // compile-time constant
```

15.2 Factory Constructors

A factory constructor can return any instance -- including a cached one, a subtype, or one from a pool. Unlike regular constructors, it does not have to create a new object every time. Common uses: singleton pattern, JSON deserialization, and object pooling.

```
// Singleton via factory -- same name returns the same object:
class Logger {
  // Cache: name -> Logger instance
  static final Map<String, Logger> _cache = {};
```

```
  final String name;

  // Private constructor -- only this class can call it:
  Logger._internal(this.name);

  // Factory returns cached or new instance:
  factory Logger(String name) {
    // putIfAbsent: if 'name' not in cache, create and insert, then return:
    return _cache.putIfAbsent(name, () => Logger._internal(name));
  }
}

var a = Logger('auth');
var b = Logger('auth');
print(identical(a, b));   // true -- same object returned both times!

// Factory for JSON deserialization -- very common in Dart apps:
class User {
  final String name; final int age;
  User(this.name, this.age);

  factory User.fromJson(Map<String, dynamic> json) =>
    User(json['name'] as String, json['age'] as int);

  Map<String, dynamic> toJson() => {'name': name, 'age': age};
}

var user = User.fromJson({'name': 'Alice', 'age': 30});
print(user.name);   // 'Alice'
```

15.3 Const Constructors & Canonicalization

When you mark a constructor as 'const', Dart can evaluate the object at compile time. Crucially, two const objects with identical arguments are the SAME object in memory (canonicalized). This saves memory and allows const objects to be used as default parameter values.

```
class Color {
  final int r, g, b;
  // const constructor -- all fields must be final, body must be empty:
  const Color(this.r, this.g, this.b);

  // Static const values -- created at compile time:
  static const red   = Color(255, 0, 0);
  static const green = Color(0, 255, 0);
  static const blue  = Color(0, 0, 255);
}

```

```
// Two const objects with same args are IDENTICAL (one object):
const c1 = Color(255, 0, 0);
const c2 = Color(255, 0, 0);
print(identical(c1, c2));   // true -- canonicalized!
print(identical(Color.red, c1)); // true -- same object

// Runtime (non-const) objects are NOT canonicalized:
var c3 = Color(255, 0, 0);   // new object each time
print(identical(c1, c3));    // false

// Rules for const constructors:
// 1. All instance fields must be final
// 2. Constructor body must be empty (use initializer list)
// 3. All constructor arguments must be compile-time constants
// 4. Cannot call non-const constructors in initializer list
```

Chapter 16 -- Annotations & Metadata

Annotations attach metadata to code declarations. They look like @SomeName and can appear before classes, methods, fields, and parameters. The Dart ecosystem uses annotations heavily for code generation (JSON serialization, dependency injection, routing), test marking, and compiler hints. You can also define your own annotations.

16.1 Built-in Annotations

Dart provides several annotations built into the language and its core libraries. @override is the most important -- it tells the compiler you intend to override a parent method, so it errors if the method doesn't actually exist in the parent.

```
// @override -- verify that the method exists in the parent:
class Animal { void speak() => print('...'); }
class Dog extends Animal {
  @override void speak() => print('Woof!');
  // Without @override, a typo like 'spek()' would silently add a new method
}

// @deprecated -- warn users that this API is outdated:
@Deprecated('Use newGreet(String name) instead. Removed in v3.0')
void oldGreet() => print('Hello!');
// Calling oldGreet() shows a warning in the IDE

// @pragma -- low-level hints to the Dart VM/compiler:
@pragma('vm:prefer-inline')  // hint: inline this function for speed
int fastAdd(int a, int b) => a + b;

@pragma('vm:never-inline')   // hint: never inline (e.g. for profiling)
void complexOperation() { /* ... */ }

// @visibleForTesting -- allows access from test files only:
@visibleForTesting
void resetState() { /* internal state reset */ }

// @nonVirtual -- prevent subclasses from overriding:
@nonVirtual
void criticalMethod() { /* must not be overridden */ }
```

16.2 Custom Annotations

Custom annotations are simply const classes. You annotate code with them, and tools (build_runner, reflection) can read the annotations at runtime or during code generation to produce boilerplate code automatically.

```
// A custom annotation is any const class:
class Route {
  final String path;
  final List<String> methods;
  const Route(this.path, {this.methods = const ['GET']});
}

class Auth {
  final bool required;
  const Auth({this.required = true});
}

// Apply annotations to a class and its methods:
@Route('/api/users', methods: ['GET', 'POST'])
@Auth(required: true)
class UserController {
  @Route('/api/users/:id')
  void getUser(String id) { /* fetch user */ }

  @Route('/api/users', methods: ['POST'])
  @Auth(required: false)
  void createUser() { /* create user */ }
}

// Tools like build_runner scan annotations and generate code.
// Popular annotation packages:
// - json_serializable  (@JsonSerializable, @JsonKey)
// - freezed            (@freezed) -- immutable data classes
// - injectable         (@injectable, @lazySingleton)
// - auto_route         (@AutoRoute, @MaterialRoute)
// Run: dart run build_runner build --delete-conflicting-outputs
```

Chapter 17 -- Error & Exception Handling

Dart distinguishes two categories of throwable objects: Errors (programming mistakes that should not be caught -- things like out-of-bounds access, null dereference, stack overflow) and Exceptions (expected failure conditions that your code should catch and recover from -- like failed parsing or network timeouts). Understanding this distinction helps you write more robust error handling.

17.1 Error vs Exception Hierarchy

The rule of thumb: catch Exceptions, not Errors. An Error means your code has a bug -- you should fix the bug, not catch the Error. An Exception means something expected went wrong that your program can recover from (bad user input, network failure, missing file).

Class	Type	When & Why It Occurs
`Error`	`Error`	Base for all programmer mistakes. Do NOT catch these.
`AssertionError`	`Error`	assert(condition) failed in debug mode. Fix the code.
`TypeError`	`Error`	Runtime type cast (as) failed. Indicates a design problem.
`StateError`	`Error`	Object used in invalid state (e.g. reading from a closed stream)
`UnsupportedError`	`Error`	Calling an unsupported operation (e.g. mutating an unmodifiable list)
`UnimplementedError`	`Error`	Abstract or missing method was called
`StackOverflowError`	`Error`	Infinite recursion -- call stack exhausted
`OutOfMemoryError`	`Error`	Heap memory exhausted
`Exception`	`Exception`	Base for recoverable errors. Catch these.
`FormatException`	`Exception`	int.parse('abc') -- input does not match expected format
`ArgumentError`	`Exception`	Function received an invalid argument value
`RangeError`	`Exception`	List index out of bounds or value outside allowed range
`NullThrownError`	`Exception`	null was explicitly thrown (rare but valid)
`IOException`	`Exception`	File system or network operation failed
`TimeoutException`	`Exception`	Async operation exceeded its time limit
`HttpException`	`Exception`	HTTP request failed (dart:io)

17.2 Custom Exceptions & Patterns

Creating custom exception classes lets you carry meaningful context (which field failed, what value was invalid) and catch specific errors precisely. The Result type pattern is an alternative approach that avoids throwing altogether -- perfect for expected failures.

```
// Custom exception with meaningful context:
class ValidationException implements Exception {
  final String field;
```

```
  final String message;
  const ValidationException(this.field, this.message);
  @override String toString() => 'ValidationException[\$field]: \$message';
}

// Multi-exception handling -- most specific first:
try {
  var n = int.parse('abc');       // FormatException
  var lst = [1,2,3];
  print(lst[10]);                  // RangeError
} on FormatException catch (e) {  // catches FormatException specifically
  print('Format: \$e');
} on RangeError catch (e, stack) {
  print('Range: \$e');
  print(stack);   // full stack trace
} catch (e) {      // catches any remaining exception
  print('Other: \$e');
} finally {         // always runs, even if rethrown
  print('Cleanup');
}

// rethrow -- re-throws keeping original stack trace (not: throw e)
void safeProcess() {
  try { riskyOp(); }
  catch (e) { log(e); rethrow; }  // log then let it propagate
}

// Result type -- functional error handling without exceptions:
sealed class Result<T> {}
class Ok<T>  extends Result<T> { final T value; Ok(this.value); }
class Err<T> extends Result<T> { final String msg; Err(this.msg); }

Result<int> divide(int a, int b) =>
  b == 0 ? Err('Division by zero') : Ok(a ~/ b);

var r = divide(10, 2);
print(switch(r) { Ok(:var value) => value, Err(:var msg) => msg });
```

Chapter 18 -- Isolates & Concurrency

Dart is single-threaded by design -- there is no shared-memory concurrency. This eliminates data races and makes async code simpler. For true parallelism (using multiple CPU cores), Dart uses Isolates -- completely isolated threads with their own memory heap that communicate only via message passing. The event loop drives all async operations.

18.1 The Event Loop

Understanding Dart's event loop explains why async code behaves the way it does. There are two queues: the microtask queue (high priority, for continuation of async/await) and the event queue (normal priority, for I/O callbacks, timers, and Future completions). Synchronous code always runs first, then microtasks, then events.

```
import 'dart:async';

void main() {
  // 1. Synchronous code runs first -- no delay:
  print('1 - Start (sync)');

  // 3. Future goes to event queue (macrotask queue):
  Future(() => print('4 - Future callback (event queue)'));

  // 2. Microtasks run BEFORE events:
  scheduleMicrotask(() => print('3 - Microtask (high priority)'));

  print('2 - End of sync code');
}
// Execution order: 1, 2, 3, 4 -- even though '4' was scheduled before '3'!

// async/await uses microtasks internally:
Future<void> example() async {
  print('A');
  await Future.value(42);  // suspends here, resumes as microtask
  print('C');  // runs before any other event queue items
}
print('B');  // prints before 'C' if example() is called without await

// Timer -- scheduled in event queue after duration:
Timer(Duration(seconds: 1), () => print('After 1 second'));
Timer.periodic(Duration(seconds: 5), (timer) {
  print('Every 5 seconds');
  if (isDone) timer.cancel();
});
```

18.2 Isolate.run -- Easy Parallelism

Isolate.run() (Dart 2.19+) is the simplest way to run heavy computation on a separate thread. The closure runs in a new isolate, then the result is returned. The closure must not capture non-sendable objects (functions, most class instances with closures).

```
import 'dart:isolate';

// Without isolates -- blocks the main thread!
int slowSum() {
  int sum = 0;
  for (int i = 0; i < 100000000; i++) sum += i;
  return sum;
}

void main() async {
  print('Starting computation...');

  // Isolate.run -- runs on a separate thread, non-blocking:
  final result = await Isolate.run(() {
    // This code runs in a new isolate with its own heap:
    int sum = 0;
    for (int i = 0; i < 100000000; i++) sum += i;
    return sum;  // result is sent back via message passing
  });

  print('Sum: \$result');   // 4999999950000000
  // Main thread was NOT blocked during computation!
}

// Rules for Isolate data transfer:
// ALLOWED: int, double, String, bool, null, List, Map, Set of these
// NOT ALLOWED: functions/closures, most class instances with closures
// NOTE: Data is COPIED between isolates, never shared
```

Chapter 19 -- Dart Core Libraries

Dart ships with a rich standard library. dart:core is imported automatically and provides fundamental types (DateTime, Duration, RegExp, etc.). dart:math provides math functions and random number generation. dart:convert handles encoding/decoding (JSON, Base64, UTF-8). dart:collection provides specialized collection types. dart:io provides file system and network access (not available in web/Flutter).

19.1 dart:core -- DateTime, Duration, RegExp

DateTime represents a specific point in time. Duration represents a span of time. RegExp provides regular expression matching for powerful text processing. These are auto-imported with dart:core.

```
// Duration -- represents a span of time:
var oneHour  = Duration(hours: 1);
var mixed    = Duration(hours: 2, minutes: 30, seconds: 15);
print(mixed.inMinutes);    // 150   -- total in minutes
print(mixed.inSeconds);    // 9015  -- total in seconds

// DateTime -- a specific moment in time:
var now     = DateTime.now();          // current local time
var utcNow  = DateTime.now().toUtc(); // current UTC time
var d       = DateTime(2025, 6, 15, 14, 30);  // year,month,day,h,m
print(now.year);
print(now.add(Duration(days: 30)));   // 30 days later
print(now.subtract(Duration(hours: 5))); // 5 hours ago
print(d.difference(now).inDays.abs()); // days between d and now
print(now.isBefore(d));               // true if now is earlier
print(now.toIso8601String());         // '2025-01-15T10:30:00.000'

// RegExp -- regular expression pattern matching:
var digits = RegExp(r'\d+');          // one or more digits
print(digits.hasMatch('abc123'));      // true
print(digits.firstMatch('abc123')?.group(0)); // '123'

// allMatches -- find all occurrences:
var all = digits.allMatches('1a2b3c');
print(all.map((m) => m.group(0)).toList()); // ['1','2','3']

// Replace with RegExp:
print('  hello  world  '.trim());          // 'hello  world'
print('hello'.replaceAll(RegExp(r'l'), 'r')); // 'herro'
```

19.2 dart:math

The math library provides constants (pi, e), common math functions (sqrt, pow, log, trig), and a Random class. import 'dart:math' to use it.

```
import 'dart:math';

// Mathematical constants:
print(pi);   // 3.141592653589793 -- ratio of circumference to diameter
print(e);    // 2.718281828459045 -- Euler's number (base of natural log)

// Power and roots:
print(pow(2, 10));    // 1024    -- 2 to the power of 10
print(sqrt(144));     // 12.0    -- square root
print(pow(27, 1/3));  // 3.0     -- cube root (via fractional exponent)

// Logarithms:
print(log(e));           // 1.0   -- natural log (ln)
print(log(100)/log(10)); // 2.0   -- log base 10 (no built-in log10)
print(log(8)/log(2));    // 3.0   -- log base 2

// Trigonometry (angles in radians):
print(sin(pi / 2));   // 1.0  -- sin of 90 degrees
print(cos(pi));       // -1.0 -- cos of 180 degrees
print(tan(pi / 4));   // 1.0  -- tan of 45 degrees
double toRad(double deg) => deg * pi / 180;  // degrees to radians

// Min/Max:
print(min(3, 7));    // 3 -- works for int and double
print(max(3, 7));    // 7

// Rounding:
print(3.7.round());    // 4 -- rounds to nearest
print(3.7.floor());    // 3 -- always rounds down
print(3.2.ceil());     // 4 -- always rounds up
print(3.9.truncate()); // 3 -- drops decimal part

// Random number generation:
var rng = Random();
print(rng.nextInt(100));    // random int 0 to 99
print(rng.nextDouble());    // random double 0.0 to 1.0
print(rng.nextBool());      // random true or false
var secure = Random.secure(); // cryptographically secure RNG
```

19.3 dart:convert

The convert library provides encoders and decoders for common formats. jsonEncode/jsonDecode handle JSON. base64Encode/base64Decode handle Base64. utf8.encode/decode handle UTF-8 byte conversion. These are used constantly in HTTP clients and data serialization.

```
import 'dart:convert';

// JSON serialization -- encode Dart objects to JSON string:
var user = {'name': 'Alice', 'age': 30, 'scores': [90, 85, 92]};
String json = jsonEncode(user);
print(json);
// '{"name":"Alice","age":30,"scores":[90,85,92]}'

// JSON deserialization -- decode JSON string to Dart object:
Map<String, dynamic> decoded = jsonDecode(json);
print(decoded['name']);    // 'Alice'
print((decoded['scores'] as List).length);  // 3

// Pretty-print JSON with indentation:
var pretty = JsonEncoder.withIndent('  ').convert(user);
print(pretty);  // multi-line formatted JSON

// Base64 encoding -- useful for embedding binary data in text:
String b64 = base64Encode(utf8.encode('Hello, Dart!'));
print(b64);   // 'SGVsbG8sIERhcnQh'
String back = utf8.decode(base64Decode(b64));
print(back);  // 'Hello, Dart!'

// UTF-8 bytes -- useful for network/file I/O:
List<int> bytes = utf8.encode('Dart');
print(bytes);  // [68, 97, 114, 116]
print(utf8.decode(bytes));  // 'Dart'
```

19.4 dart:collection -- Specialized Collections

dart:collection provides specialized implementations of List, Set, and Map with different performance characteristics or ordering guarantees. Choose the right type for your use case to optimize performance and correctness.

```
import 'dart:collection';

// Queue -- optimized for adding/removing at both ends O(1):
// Use instead of List when you need a FIFO or deque structure
var q = Queue<int>();
q.addLast(1);   q.addLast(2);   q.addFirst(0);  // [0, 1, 2]
print(q.removeFirst());   // 0 -- O(1) unlike List.removeAt(0)
print(q.removeLast());    // 2

// SplayTreeMap -- sorted by key, good for range queries:
var sorted = SplayTreeMap<String, int>();
sorted['b'] = 2; sorted['a'] = 1; sorted['c'] = 3;
print(sorted.keys.toList());   // ['a', 'b', 'c'] -- always sorted
```

```
print(sorted.firstKey());      // 'a' -- O(log n)
print(sorted.lastKey());       // 'c' -- O(log n)

// SplayTreeSet -- sorted unique elements:
var ts = SplayTreeSet<int>.from([3, 1, 4, 1, 5, 2]);
print(ts.toList());  // [1, 2, 3, 4, 5] -- sorted, deduplicated

// LinkedHashSet -- preserves insertion order (unlike regular Set):
var lhs = LinkedHashSet<int>.from([3, 1, 4, 1, 5]);
print(lhs.toList());  // [3, 1, 4, 5] -- insertion order, deduplicated

// HashMap -- fastest Map, no order guarantee (just like HashSet):
var hm = HashMap<String, int>({'a': 1, 'b': 2, 'c': 3});
// Good when you only need fast lookup and don't care about order
```

Chapter 20 -- StringBuffer & Advanced Strings

20.1 StringBuffer -- Efficient String Building

String concatenation with + is fine for a few operations but becomes O(n^2) in a loop because each + creates a new string object. StringBuffer solves this by accumulating characters in a buffer and only creating the final string once at the end -- O(n) total. Always use StringBuffer when building strings in loops.

```
// SLOW: each += creates a new String object -- O(n^2) for n items:
String slow = '';
for (int i = 0; i < 1000; i++) {
  slow += 'item$i,';  // creates 1000 intermediate String objects!
}

// FAST: StringBuffer accumulates, single String created at end -- O(n):
var sb = StringBuffer();
for (int i = 0; i < 1000; i++) {
  sb.write('item');
  sb.write(i);
  sb.write(',');
}
String fast = sb.toString();  // create String only once

// StringBuffer methods:
var buf = StringBuffer();
buf.write('Hello');              // adds text, no newline
buf.writeln(', World!');         // adds text + newline
buf.writeAll(['a','b','c'],'-');// 'a-b-c'
buf.write(42);                   // numbers auto-converted
buf.write(true);                 // booleans auto-converted
print(buf.length);               // current character count
print(buf.isEmpty);              // true if nothing written
buf.clear();                     // reset to empty
print(buf.toString());           // build the final String

// Common use: building HTML, CSV, SQL, etc.:
var csv = StringBuffer('name,age,score\n');
for (var user in users) {
  csv.write('${user.name},${user.age},${user.score}\n');
}
File('data.csv').writeAsString(csv.toString());
```

20.2 Advanced Regular Expressions

RegExp in Dart supports capture groups (parentheses), named groups, flags (case-insensitive, multiLine), and the replaceAllMapped method for replacing with a function. Understanding groups lets you extract specific parts of a match.

```
// Capture groups -- parentheses create numbered groups:
var dateRe = RegExp(r'(\d{4})-(\d{2})-(\d{2})');  // yyyy-mm-dd
var m = dateRe.firstMatch('Today is 2025-06-15');
print(m?.group(0));  // '2025-06-15' -- group(0) is always whole match
print(m?.group(1));  // '2025'  -- first capture group
print(m?.group(2));  // '06'    -- second
print(m?.group(3));  // '15'    -- third

// Named groups -- cleaner than numbered groups:
var named = RegExp(r'(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})');
var nm = named.firstMatch('2025-06-15');
print(nm?.namedGroup('year'));   // '2025'
print(nm?.namedGroup('month'));  // '06'

// Flags:
var caseRe = RegExp(r'hello', caseSensitive: false);
print(caseRe.hasMatch('HELLO'));   // true

var multiRe = RegExp(r'^\w+', multiLine: true); // ^ matches each line start

// replaceAllMapped -- replace each match using a function:
String result = 'camelCaseString'.replaceAllMapped(
  RegExp(r'[A-Z]'),
  (m) => '_${m.group(0)!.toLowerCase()}',
);
print(result);   // 'camel_case_string'

// Find all email addresses in text:
var emailRe = RegExp(r'[\w.]+@[\w]+\.[a-zA-Z]+');
var text = 'Contact: a@x.com and b@y.org';
var emails = emailRe.allMatches(text).map((m) => m.group(0)).toList();
print(emails);   // ['a@x.com', 'b@y.org']
```

Chapter 21 -- Advanced Async: Futures & Streams

Beyond basic async/await, Dart provides powerful tools for coordinating multiple asynchronous operations: Future combinators (wait, any, timeout) for running Futures in parallel, Stream operations for transforming async sequences, and StreamController for manually pushing events into a stream.

21.1 Future Combinators

These static methods on Future let you orchestrate multiple async operations efficiently. Future.wait runs them in parallel and collects all results. Future.any returns the first to complete. .timeout() fails the Future if it takes too long.

```
// Future.wait -- run multiple Futures IN PARALLEL:
// All three requests start at the same time!
var results = await Future.wait([
  fetchUser(1),      // starts immediately
  fetchUser(2),      // starts immediately
  fetchProfile(42),  // starts immediately
]);
// Total time = longest individual time, not sum of all
print(results);   // [user1, user2, profile42]

// Future.any -- first to complete wins, others are discarded:
var fastest = await Future.any([
  fetchFromPrimaryServer(),   // race between them
  fetchFromBackupServer(),
]);
// Good for redundant data sources

// .timeout() -- fail if operation takes too long:
try {
  var data = await fetchData()
    .timeout(Duration(seconds: 5));   // fails after 5 seconds
} on TimeoutException {
  print('Request timed out -- try again later');
}

// Future.value / Future.error -- immediately completed Futures:
Future<int> getAnswer() => Future.value(42);     // already resolved
Future<int> getFail() => Future.error('Oops!');  // already failed

// Chaining with .then() (callback style):
fetchUser(1)
  .then((user) => fetchPosts(user.id))
  .then((posts) => print(posts.length))
```

```
  .catchError((e) => print('Error: $e'))
  .whenComplete(() => print('Done'));
```

21.2 Stream Operations

Streams support the same functional operations as Iterable but asynchronously. All transformations (where, map, take) are lazy and applied as data flows through. You can also convert a Stream to a Future using methods like .first, .toList(), .reduce().

```
// Create streams:
Stream<int> countUp(int to) async* {
  for (int i = 1; i <= to; i++) {
    await Future.delayed(Duration(milliseconds: 100));
    yield i;
  }
}

// Transform stream lazily -- no data flows yet:
Stream<String> result = countUp(10)
  .where((n) => n.isEven)              // filter: keep even numbers
  .map((n) => 'Even: $n')             // transform: to strings
  .take(3);                            // stop after 3 items

// Consume with await for:
await for (String s in result) {
  print(s);   // 'Even: 2', 'Even: 4', 'Even: 6'
}

// Aggregate stream to single value:
int sum = await countUp(5).reduce((a, b) => a + b);  // 15
List<int> all = await countUp(5).toList();   // [1,2,3,4,5]
bool any = await countUp(5).any((n) => n > 3);  // true
int first = await countUp(5).first;  // 1

// Stream error handling:
await for (var n in riskyStream()) {
  try { process(n); }
  catch (e) { print('Skipping bad value: $e'); }
}
```

21.3 StreamController

A StreamController lets you manually create streams and push events into them. The 'sink' is where you add data; the 'stream' is what listeners subscribe to. Broadcast controllers allow multiple listeners. This pattern is used extensively in state management (BLoC pattern uses StreamControllers).

```dart
import 'dart:async';

// Single-subscription StreamController (default):
// Only ONE listener at a time. Good for file reads, HTTP responses.
var sc = StreamController<int>();

// Push events via sink:
sc.sink.add(1);
sc.sink.add(2);
sc.sink.add(3);
sc.sink.addError(FormatException('Invalid data'));
sc.sink.close();   // signal: no more events

// Listen to the stream:
sc.stream.listen(
  (data) => print('Data: $data'),
  onError: (e) => print('Error: $e'),
  onDone: () => print('Stream closed'),
);

// Broadcast StreamController -- multiple listeners allowed:
// Good for event buses, state updates, UI notifications.
var bc = StreamController<String>.broadcast();
bc.stream.listen((s) => print('Listener 1: $s'));
bc.stream.listen((s) => print('Listener 2: $s'));
bc.sink.add('Hello!');   // Both listeners receive 'Hello!'
bc.close();

// Practical: event bus pattern:
class EventBus {
  final _controller = StreamController<dynamic>.broadcast();
  Stream<T> on<T>() => _controller.stream.whereType<T>();
  void emit(dynamic event) => _controller.add(event);
  void dispose() => _controller.close();
}
```

Chapter 22 -- Generics (Advanced)

Advanced generics cover bounded type parameters (restricting which types T can be), generic methods on non-generic classes, variance (how List<Dog> relates to List<Animal>), and reified generics (Dart preserves type information at runtime, unlike Java's type erasure).

22.1 Bounded Type Parameters

Adding 'extends SomeType' to a type parameter restricts which types can be used. This lets you safely call methods of SomeType on values of T without a cast, because the compiler knows T is at least SomeType.

```
// Without bound: T could be anything -- can only call Object methods:
class SimpleBox<T> {
  T value;
  SimpleBox(this.value);
  // Cannot call .compareTo() -- T might not have it
}

// With bound: T must be Comparable -- can safely call .compareTo():
class SortedList<T extends Comparable<T>> {
  final List<T> _data = [];

  void add(T item) {
    _data.add(item);
    _data.sort();   // T has .compareTo() -- safe to sort!
  }

  T get min => _data.first;
  T get max => _data.last;
  List<T> get sorted => List.unmodifiable(_data);
}

var ints = SortedList<int>();
ints.add(3); ints.add(1); ints.add(4); ints.add(1);
print(ints.sorted);  // [1, 1, 3, 4]
print(ints.min);     // 1
print(ints.max);     // 4

// Works for any Comparable type:
var strs = SortedList<String>();
strs.add('banana'); strs.add('apple'); strs.add('cherry');
print(strs.min);  // 'apple'
```

22.2 Dart Generics are Reified

Unlike Java, Dart preserves type information at runtime. You can check the type of a generic at runtime, which makes some patterns much simpler. However, List<Dog> is covariant (assignable to List<Animal>) by default -- this is convenient but can cause runtime errors if misused.

```
// Dart preserves generic type info at runtime (reified generics):
List<int>    ints    = [1, 2, 3];
List<String> strings = ['a', 'b'];

print(ints is List<int>);      // true
print(ints is List<String>);   // false -- runtime check works!
print(ints.runtimeType);       // 'List<int>'

// Covariance -- List<Dog> can be assigned to List<Animal>:
class Animal {}
class Dog extends Animal {}
class Cat extends Animal {}

List<Dog> dogs = [Dog(), Dog()];
List<Animal> animals = dogs;   // OK -- covariance
// BUT: adding a Cat to 'animals' throws at runtime:
// animals.add(Cat());  // throws: type 'Cat' is not a subtype of 'Dog'

// For type-safe collections, use List.unmodifiable or avoid aliasing:
List<Animal> safeAnimals = List<Animal>.from(dogs); // copy -- no aliasing
safeAnimals.add(Cat());  // OK -- safeAnimals is really List<Animal>

// Type inference -- Dart infers generic types from context:
var nums = [1, 2, 3];          // inferred List<int>
var map  = {'a': 1, 'b': 2};  // inferred Map<String, int>
var firstNum = nums.first;     // inferred int
```

Chapter 23 -- List Methods (Complete Reference)

A List<T> is Dart's general-purpose ordered collection. Elements are accessed by zero-based index in O(1). Appending (add) is amortized O(1). Insertion and removal at arbitrary positions is O(n) because elements must shift. Lists are growable by default -- they resize automatically.

23.1 Creating Lists

There are several ways to create a List. The literal syntax is most common. List.filled creates a fixed-size list pre-filled with a value. List.generate creates a list by applying a function to each index -- ideal for computed sequences.

```
// Literal -- most common, infers type from elements:
List<int> nums = [1, 2, 3, 4, 5];
var fruits = <String>['apple', 'banana', 'cherry'];

// List.filled -- fixed-size, every element gets the same value:
var zeros = List.filled(5, 0);          // [0, 0, 0, 0, 0]
var falses = List.filled(3, false);     // [false, false, false]

// List.generate -- compute each element from its index:
var squares = List.generate(6, (i) => i * i);  // [0,1,4,9,16,25]
var strs    = List.generate(3, (i) => 'item$i');// ['item0','item1','item2']

// Spread operator -- unpack another list into a new list:
var a = [1, 2, 3];
var b = [0, ...a, 4];   // [0, 1, 2, 3, 4]

// Null-aware spread -- safe when the source might be null:
List<int>? maybeList = null;
var safe = [0, ...?maybeList, 99];   // [0, 99] -- null is skipped

// Empty list:
var empty = <String>[];          // explicit type
var empty2 = List<int>.empty(growable: true);
```

23.2 Adding & Inserting Elements

add() appends to the end in O(1). insert() places an element before a given index but is O(n) because all subsequent elements must shift right. addAll() and insertAll() are bulk versions of these operations.

Method / Property	Example	Description & Return Value
`lst.add(v)`	`lst.add(99)`	Append to END. Returns void. Amortized O(1)
`lst.addAll(iterable)`	`lst.addAll([4,5,6])`	Append ALL elements from iterable. Returns void

Method / Property	Example	Description & Return Value
`lst.insert(i, v)`	`lst.insert(0, 'a')`	Insert v BEFORE index i. Shifts elements right. O(n)
`lst.insertAll(i, it)`	`lst.insertAll(1,[2,3])`	Insert all elements from iterable before index i
`lst[i] = value`	`lst[0] = 99`	Overwrite element at index i. O(1). RangeError if out of bounds
`lst.setAll(i, iter)`	`lst.setAll(1, [7,8])`	Overwrite elements starting at index i. Does not resize list
`lst.setRange(s,e,it)`	`lst.setRange(0,2,src)`	Overwrite range [s,e) from iterable. e is exclusive
`lst.fillRange(s,e,v)`	`lst.fillRange(0,3,0)`	Fill range [s,e) with constant value v

23.3 Removing Elements

remove() searches for and removes the first matching value -- O(n). removeAt() removes by index -- also O(n) because elements shift. removeLast() is O(1). removeWhere() and retainWhere() filter the list in-place based on a predicate.

Method / Property	Example	Description & Return Value
`lst.remove(v)`	`lst.remove(3)`	Remove FIRST occurrence of v. Returns true if found. O(n)
`lst.removeAt(i)`	`lst.removeAt(0)`	Remove element at index i and return it. Shifts elements. O(n)
`lst.removeLast()`	`lst.removeLast()`	Remove and return the last element. O(1)
`lst.removeRange(s,e)`	`lst.removeRange(1,3)`	Remove all elements from index s to e (exclusive). O(n)
`lst.removeWhere(t)`	`lst.removeWhere((x)=>x<0)`	Remove all elements where predicate returns true. O(n)
`lst.retainWhere(t)`	`lst.retainWhere((x)=>x>0)`	Keep only elements where predicate returns true. O(n)
`lst.clear()`	`lst.clear()`	Remove ALL elements. Sets length to 0. O(n)

```
List<int> lst = [1, 2, 3, 2, 4, 5];
print(lst.remove(2));      // true  -- removes first 2: [1,3,2,4,5]
print(lst.remove(99));     // false -- 99 not found
print(lst.removeAt(0));    // 1     -- removes at index 0: [3,2,4,5]
print(lst.removeLast());   // 5     -- removes last: [3,2,4]

List<int> data = [1,-2,3,-4,5,-6];
data.removeWhere((x) => x < 0);  // [1, 3, 5]
data.retainWhere((x) => x > 2);  // [3, 5]
data.clear();                    // []
```

23.4 Searching & Accessing

Direct index access (lst[i]) is O(1). contains() and indexOf() are O(n) linear searches. indexWhere() finds the first index matching a predicate. These are powerful for finding elements by condition rather than by value.

Method / Property	Example	Description & Return Value
`lst[i]`	`lst[0]`	Access element at index. O(1). RangeError if out of bounds
`lst.first`	`lst.first`	First element. Throws StateError if list is empty
`lst.last`	`lst.last`	Last element. Throws StateError if list is empty
`lst.contains(v)`	`lst.contains(5)`	True if value exists. O(n) linear scan
`lst.indexOf(v,s)`	`lst.indexOf(3)`	First index of v, starting at s. Returns -1 if not found
`lst.lastIndexOf(v,s)`	`lst.lastIndexOf(3)`	Last index of v. Returns -1 if not found
`lst.indexWhere(t,s)`	`lst.indexWhere((x)=>x>3)`	First index where predicate is true. Returns -1 if none
`lst.lastIndexWhere(t)`	`lst.lastIndexWhere((x)=>x<3)`	Last index where predicate is true
`lst.sublist(s,e)`	`lst.sublist(1,4)`	New list from index s to e (exclusive). Both ends are optional

23.5 Transformation & Query

These methods apply functional-style operations. map(), where(), and expand() return lazy Iterables -- they do not evaluate immediately. Call .toList() to get a concrete List. reduce() and fold() aggregate all elements into one value.

Method / Property	Example	Description & Return Value
`lst.map(f)`	`lst.map((x)=>x*2)`	Transform each element. Returns LAZY Iterable<R>
`lst.where(test)`	`lst.where((x)=>x>3)`	Filter: keep elements where test is true. Lazy Iterable
`lst.expand(f)`	`lst.expand((x)=>[x,x])`	FlatMap: map to Iterable then flatten one level. Lazy
`lst.reduce(f)`	`lst.reduce((a,b)=>a+b)`	Combine all elements into one. Throws StateError if empty
`lst.fold(init,f)`	`lst.fold(0,(a,b)=>a+b)`	Like reduce but with initial value. Works on empty lists
`lst.any(test)`	`lst.any((x)=>x>4)`	True if ANY element passes test. Short-circuits
`lst.every(test)`	`lst.every((x)=>x>0)`	True if ALL elements pass test. Short-circuits
`lst.firstWhere(t)`	`lst.firstWhere((x)=>x>2)`	First element passing test. Use orElse: for safety
`lst.lastWhere(t)`	`lst.lastWhere((x)=>x>2)`	Last element passing test
`lst.take(n)`	`lst.take(3)`	Lazy Iterable of first n elements
`lst.skip(n)`	`lst.skip(2)`	Lazy Iterable skipping first n elements
`lst.join(sep)`	`lst.join(', ')`	Concatenate all elements as a String with separator
`lst.toSet()`	`lst.toSet()`	Convert to Set -- removes duplicate values
`lst.reversed`	`lst.reversed.toList()`	Lazy reversed view. Original list is NOT modified
`lst.asMap()`	`lst.asMap()`	Returns Map<int,E> mapping index to element

23.6 Sorting & Reordering

sort() modifies the list IN-PLACE and returns void. The optional comparator function lets you define custom sort order. A comparator returns negative if a < b, 0 if equal, positive if a > b. shuffle() randomizes the order in place.

```
List<int> lst = [3, 1, 4, 1, 5, 9, 2];

// Default sort -- ascending (uses natural ordering):
lst.sort();   // [1, 1, 2, 3, 4, 5, 9]

// Custom comparator -- descending order:
lst.sort((a, b) => b - a);   // [9, 5, 4, 3, 2, 1, 1]
// Comparator: negative = a comes first, 0 = equal, positive = b comes first

// Sort by a property:
List<String> words = ['banana', 'apple', 'fig', 'cherry'];
words.sort((a, b) => a.length.compareTo(b.length));
// ['fig', 'apple', 'banana', 'cherry']  -- sorted by string length

// Non-destructive reverse (original unchanged):
var rev = lst.reversed.toList();

// Functional pipeline example:
var result = [5, 2, 8, 1, 9, 3, 7, 4, 6]
  .where((x) => x > 3)           // [5, 8, 9, 7, 4, 6]
  .map((x) => x * x)             // [25, 64, 81, 49, 16, 36]
  .toList()..sort();              // [16, 25, 36, 49, 64, 81]
print(result);
```

Chapter 24 -- Set Methods (Complete Reference)

A Set<T> is an unordered collection with a guarantee: every element is unique. Dart's default Set is a LinkedHashSet, which preserves insertion order while still providing O(1) average for add/remove/contains. Use a Set when you need fast membership tests or need to eliminate duplicates.

24.1 Creating Sets

Set literals use curly braces {}. IMPORTANT: an empty {} is an empty Map, not an empty Set. Use <Type>{} or Set<Type>() for empty sets. From dart:collection, SplayTreeSet gives sorted order and LinkedHashSet preserves insertion order.

```
// Set literal -- duplicates silently ignored:
Set<int> nums = {1, 2, 3, 2, 1};   // {1, 2, 3}
var fruits = <String>{'apple', 'banana', 'apple'};  // {apple, banana}

// From List -- fastest way to deduplicate:
var deduped = [1, 2, 2, 3, 3, 3].toSet();  // {1, 2, 3}

// IMPORTANT: {} is empty Map, NOT empty Set!
var emptyMap = {};           // Map<dynamic,dynamic> -- Map!
var emptySet = <int>{};      // Set<int>   -- correct
var emptySet2 = Set<int>();  // Set<int>   -- also correct

// Specialized Set types from dart:collection:
import 'dart:collection';
// SplayTreeSet -- maintains sorted order, O(log n) operations:
var sorted = SplayTreeSet<int>.from([3,1,4,1,5]);  // {1,3,4,5}
// LinkedHashSet -- preserves insertion order (default Set behavior):
var ordered = LinkedHashSet<int>.from([3,1,2]);    // {3,1,2}
// HashSet -- fastest, NO order guarantee:
var fast = HashSet<int>.from([3,1,2]);
```

24.2 Adding & Removing Elements

add() returns a boolean -- true if the element was actually added (not already present), false if it was a duplicate. This makes Sets great for deduplication in loops. remove() also returns bool.

Method / Property	Example	Description & Return Value
`s.add(v)`	`s.add(4)`	Add v. Returns TRUE if added, FALSE if already existed. O(1)
`s.addAll(iter)`	`s.addAll([5,6])`	Add all elements from iterable. Duplicates silently ignored

Method / Property	Example	Description & Return Value
`s.remove(v)`	`s.remove(3)`	Remove v. Returns TRUE if found/removed, FALSE if not found. O(1)
`s.removeAll(iter)`	`s.removeAll([1,2])`	Remove all elements that appear in iterable. O(n)
`s.retainAll(iter)`	`s.retainAll([3,4])`	Remove all elements NOT in iterable (in-place intersection)
`s.removeWhere(t)`	`s.removeWhere((x)=>x<0)`	Remove all elements where test returns true
`s.retainWhere(t)`	`s.retainWhere((x)=>x>0)`	Keep only elements where test returns true
`s.clear()`	`s.clear()`	Remove all elements from set

```
Set<int> s = {1, 2, 3};
print(s.add(4));     // true  -- 4 was new: {1,2,3,4}
print(s.add(2));     // false -- 2 already in set, no change
print(s.remove(3)); // true  -- 3 was found and removed: {1,2,4}
print(s.remove(99));// false -- 99 not in set

s.addAll([5, 6, 1]);      // {1,2,4,5,6} -- 1 is duplicate, skipped
s.removeAll([1, 2]);      // {4,5,6}
s.retainAll([4, 5]);      // {4,5} -- 6 removed
s.removeWhere((x) => x > 4);  // {4}
```

24.3 Set Operations

These operations implement classic mathematical set theory. They all return NEW sets -- neither original is modified. These operations make Set the right tool for tasks like finding common items, unique items, or combining collections.

Method / Property	Example	Description & Return Value
`s.union(other)`	`a.union(b)`	All elements in a OR b (or both). No duplicates in result
`s.intersection(other)`	`a.intersection(b)`	Only elements present in BOTH a AND b
`s.difference(other)`	`a.difference(b)`	Elements in a but NOT in b. Order matters!
`s.contains(v)`	`s.contains(3)`	True if v is in set. O(1) average -- much faster than List!
`s.containsAll(iter)`	`s.containsAll([1,2])`	True if ALL elements of iterable are in this set
`s.lookup(v)`	`s.lookup(obj)`	Return the actual element that equals v, or null. Useful for canonical objects

```
Set<int> a = {1, 2, 3, 4};
Set<int> b = {3, 4, 5, 6};

print(a.union(b));          // {1,2,3,4,5,6} -- all elements
print(a.intersection(b));   // {3,4}          -- common elements only
print(a.difference(b));     // {1,2}          -- in a, NOT in b
print(b.difference(a));     // {5,6}          -- in b, NOT in a
```

```
// Symmetric difference -- elements in EITHER but not BOTH:
var sym = a.difference(b).union(b.difference(a));
print(sym);   // {1,2,5,6}

// Practical use -- fast deduplication:
var items = [1,3,2,1,5,3,2,4];
var unique = items.toSet().toList();  // [1,3,2,5,4] -- order preserved!

// Membership test -- much faster than List.contains for large sets:
var validCodes = {'USD', 'EUR', 'GBP', 'JPY'};
print(validCodes.contains('EUR'));   // true -- O(1)
```

Chapter 25 -- Map Methods (Complete Reference)

A Map<K,V> stores key-value pairs where every key is unique. It is Dart's equivalent of a dictionary or hash table. The default Map is a LinkedHashMap -- insertion-ordered with O(1) average for get/put/remove. Maps are essential for configuration, JSON data, caching, and grouping data by a key.

25.1 Creating Maps

Map literals are the most common way to create a Map. Several static constructors help when building Maps from existing data. Map.fromIterables is especially useful for zipping two parallel lists into a Map.

```
// Map literal -- keys must be unique:
Map<String, int> scores = {'Alice': 90, 'Bob': 85, 'Carol': 75};
var config = <String, dynamic>{'host': 'localhost', 'port': 8080};

// Empty map:
var empty  = <String, int>{};
var empty2 = Map<String, int>();

// From two parallel lists (zip them together):
var keys   = ['a', 'b', 'c'];
var values = [1, 2, 3];
var m = Map.fromIterables(keys, values);   // {'a':1,'b':2,'c':3}

// From a list of MapEntry objects:
var entries = [MapEntry('x', 10), MapEntry('y', 20)];
var m2 = Map.fromEntries(entries);   // {'x':10,'y':20}

// From an iterable with computed keys/values:
var words = ['apple', 'banana', 'fig'];
var lengths = Map.fromIterable(words,
  key: (w) => w,
  value: (w) => w.length);
// {'apple':5,'banana':6,'fig':3}

// Unmodifiable (read-only) map:
var readOnly = Map.unmodifiable(scores);  // throws on write attempts
```

25.2 Accessing & Checking

Map access via m[key] returns null if the key is missing -- not an exception. This is different from List which throws RangeError. Use containsKey() to check existence before accessing, or use ?? to provide a default.

Method / Property	Example	Description & Return Value
`m[key]`	`m['name']`	Get value for key. Returns NULL if key missing (no exception)
`m[key] ?? def`	`m['x'] ?? 0`	Get with default value (Dart has no .get() method)
`m.containsKey(k)`	`m.containsKey('a')`	True if key exists in map. O(1) average
`m.containsValue(v)`	`m.containsValue(5)`	True if any entry has this value. O(n) -- scans all values
`m.keys`	`m.keys`	Iterable<K> of all keys (live view, ordered in LinkedHashMap)
`m.values`	`m.values`	Iterable<V> of all values (live view)
`m.entries`	`m.entries`	Iterable<MapEntry<K,V>> of all key-value pairs (live view)
`m.length`	`m.length`	Number of key-value pairs in the map
`m.isEmpty`	`m.isEmpty`	True if map has no entries
`m.isNotEmpty`	`m.isNotEmpty`	True if map has at least one entry

```
Map<String, int> m = {'a': 1, 'b': 2, 'c': 3};

// Access -- null if missing, NOT an exception:
print(m['a']);          // 1
print(m['x']);          // null -- key not found
print(m['x'] ?? 0);    // 0   -- safe with default

// Existence checks:
print(m.containsKey('b'));    // true
print(m.containsValue(99));   // false

// Iterate entries:
for (var entry in m.entries) {
  print('${entry.key}: ${entry.value}');
}
```

25.3 Modifying Maps

Setting m[key] = value both adds new entries and updates existing ones. putIfAbsent() inserts only if the key is absent -- very useful for building frequency maps or grouping. update() allows computing a new value from the old one.

Method / Property	Example	Description & Return Value
`m[k] = v`	`m['x'] = 10`	Add new key or overwrite existing value. O(1)
`m.addAll(other)`	`m.addAll({'d':4})`	Merge all entries from other. Later values overwrite

Method / Property	Example	Description & Return Value
`m.addEntries(iter)`	`m.addEntries([...])`	Add entries from Iterable<MapEntry<K,V>>
`m.putIfAbsent(k,fn)`	`m.putIfAbsent('x',()=>0)`	Insert only if key absent. Returns current or new value
`m.update(k,fn)`	`m.update('a',(v)=>v+1)`	Update existing value using function. Throws if key missing
`m.update(k,fn,ifAbsent:fn2)`	`m.update('x',(v)=>v+1,ifAbsent:()=>0)`	Update with fallback if key missing
`m.updateAll(fn)`	`m.updateAll((k,v)=>v*2)`	Apply function to ALL values. Returns void
`m.remove(k)`	`m.remove('a')`	Remove key and return its value. Returns null if not found
`m.removeWhere(test)`	`m.removeWhere((k,v)=>v<5)`	Remove all entries where test returns true
`m.clear()`	`m.clear()`	Remove all entries from map

```
Map<String, int> m = {'a': 1, 'b': 2};

// Add/update with []:
m['c'] = 3;           // adds new key: {a:1,b:2,c:3}
m['a'] = 99;          // updates existing: {a:99,b:2,c:3}

// putIfAbsent -- great for frequency counting:
var freq = <String, int>{};
for (var word in ['a','b','a','c','a','b']) {
  freq.update(word, (v) => v + 1, ifAbsent: () => 1);
}
print(freq);   // {a:3, b:2, c:1}

// updateAll -- transform all values:
m.updateAll((key, val) => val * 10);

// remove -- returns the removed value:
int? removed = m.remove('a');   // returns 99, m no longer has 'a'
m.removeWhere((k, v) => v > 50);  // remove expensive entries
```

25.4 Transforming Maps

Map.map() transforms all entries and returns a new Map. To filter, you must work with .entries (an Iterable of MapEntry) and then reconstruct the Map. The spread operator {...map1, ...map2} is the easiest way to merge maps.

```
Map<String, int> scores = {'Alice':90, 'Bob':60, 'Carol':75};

// map() -- transform all entries into a new Map:
var graded = scores.map((name, score) =>
  MapEntry(name, score >= 70 ? 'Pass' : 'Fail'));
// {'Alice':'Pass','Bob':'Fail','Carol':'Pass'}

// Filter -- no direct .where() on Map, use entries:
var passing = Map.fromEntries(
  scores.entries.where((e) => e.value >= 70));
// {'Alice':90, 'Carol':75}

// Sort by value (descending):
var sortedByScore = Map.fromEntries(
  scores.entries.toList()
    ..sort((a, b) => b.value.compareTo(a.value)));
// {'Alice':90,'Carol':75,'Bob':60}

// Merge maps with spread (later overwrites earlier on conflict):
var defaults = {'theme': 'light', 'lang': 'en', 'debug': false};
var userPrefs = {'theme': 'dark', 'fontSize': 14};
var merged = {...defaults, ...userPrefs};
// {'theme':'dark','lang':'en','debug':false,'fontSize':14}

// Invert map (swap keys and values):
var inverted = {for (var e in scores.entries) e.value: e.key};
// {90:'Alice', 60:'Bob', 75:'Carol'}
```

Chapter 26 -- Collection-if, Collection-for & Spread Operators

Dart's collection literals support conditional elements (collection-if), loops (collection-for), and spreading (spread operator ...). These features let you build collections declaratively -- describing WHAT you want rather than how to build it imperatively with loops. They work in List, Set, and Map literals.

26.1 Collection-if

Use 'if' inside a collection literal to conditionally include one or more elements. This replaces the pattern of creating an empty list and conditionally calling add(). It makes the final shape of a collection visible at a glance.

```
bool isPremium = true;
String env = 'production';

// Without collection-if (verbose):
List<String> old = ['basic'];
if (isPremium) old.add('premium');
if (env == 'production') old.add('ssl');

// With collection-if (declarative and readable):
List<String> features = [
  'basic',                          // always included
  if (isPremium) 'premium',         // conditional single element
  if (env == 'production') 'ssl',   // another condition
  if (!isPremium) 'ads' else 'no_ads',  // if-else form
];
// ['basic', 'premium', 'ssl', 'no_ads']

// Also works in Map literals:
Map<String, dynamic> config = {
  'host': 'localhost',
  if (env == 'production') 'ssl': true,
  if (isPremium) 'maxItems': 1000,
};

// And in Set literals:
Set<String> roles = {
  'user',
  if (isPremium) 'premium_user',
  if (isAdmin) ...{'admin', 'moderator'}, // spread inside if!
};
```

26.2 Collection-for

Use 'for' inside a collection literal to produce multiple elements from an existing iterable. This is Dart's equivalent of a list comprehension in Python. You can combine it with 'if' for filtering and nest multiple fors.

```
// Basic collection-for -- creates elements from a range:
List<int> squares = [for (int i = 1; i <= 5; i++) i * i];
// [1, 4, 9, 16, 25]

// Combined with if -- filter and transform simultaneously:
List<int> evenSquares = [
  for (int i = 1; i <= 10; i++)
    if (i.isEven) i * i,     // only square even numbers
];
// [4, 16, 36, 64, 100]

// Nested for -- Cartesian product:
List<String> pairs = [
  for (var r in ['A','B','C'])
    for (var c in [1, 2, 3])
      '$r$c',
];
// ['A1','A2','A3','B1','B2','B3','C1','C2','C3']

// Map literal with for -- build a lookup table:
Map<int, int> sqMap = {for (int i = 1; i <= 5; i++) i: i * i};
// {1:1, 2:4, 3:9, 4:16, 5:25}

// Set literal with for -- unique computed values:
Set<int> tripleSet = {for (int i in [1,2,3,2,1]) i * 3};
// {3, 6, 9} -- duplicates from 2 and 1 appearing twice removed

// Practical: transform a list of objects to a Map:
var users = [User('Alice',30), User('Bob',25)];
var nameToAge = {for (var u in users) u.name: u.age};
// {'Alice':30, 'Bob':25}
```

26.3 Spread Operators

The spread operator (...) unpacks an iterable into a collection literal. The null-aware spread (...?) safely handles null iterables by inserting nothing. Spreads work in List, Set, and Map literals, and can appear anywhere in the literal.

```
List<int> a = [1, 2, 3];
List<int> b = [4, 5, 6];

// Basic spread -- unpack into new list:
var combined = [...a, ...b];     // [1,2,3,4,5,6]
var withExtra = [0, ...a, 4];   // [0,1,2,3,4]
```

```
// Null-aware spread -- safely handles null:
List<int>? maybe = null;
var safe = [0, ...?maybe, 99];  // [0, 99] -- null is skipped

// Set spread -- combines and deduplicates:
Set<int> s = {...a, ...b};       // {1,2,3,4,5,6}

// Map spread -- merges maps (later overwrites on conflict):
var m1 = {'a': 1, 'b': 2};
var m2 = {'b': 99, 'c': 3};     // 'b' conflicts
var merged = {...m1, ...m2};    // {'a':1,'b':99,'c':3} -- m2 wins

// Practical: config merging pattern:
Map<String, dynamic> defaults = {'debug': false, 'maxRetries': 3};
Map<String, dynamic> overrides = {'debug': true};
var finalConfig = {...defaults, ...overrides};
// {'debug':true, 'maxRetries':3} -- overrides win, defaults fill rest

// Combine spread with if:
bool addExtras = true;
var lst = [
  1, 2, 3,
  if (addExtras) ...[4, 5, 6],  // spread multiple items conditionally
];
// [1, 2, 3, 4, 5, 6]
```

Chapter 27 -- Quick Comparison -- Dart Collections

Use this table to quickly choose the right collection type for your use case. The key question: do you need ORDER (use List), UNIQUENESS (use Set), or KEY-VALUE LOOKUP (use Map)? Performance characteristics guide fine-tuning once you've picked the right type.

Property	List<T>	Set<T>	Map<K,V>
Ordered by index?	Yes -- always	No (LinkedHashSet: insert)	No (LinkedHashMap: insert)
Allows duplicates?	Yes	No -- unique only	Keys: No, Values: Yes
Index access?	lst[i] O(1)	No index access	map[key] O(1) avg
Best for...	Sequences, stacks, queues	Deduplication, fast lookup	Key-value store, grouping
Add element	add(x) -- O(1) amort.	add(x) -- O(1) avg	m[key]=value -- O(1) avg
Remove by value	remove(x) -- O(n)	remove(x) -- O(1) avg	remove(key) -- O(1) avg
Remove by index	removeAt(i) -- O(n)	No index access	N/A
Membership test	contains(x) -- O(n)	contains(x) -- O(1) avg	containsKey(k) -- O(1) avg
Transform elements	map(f).toList()	map(f).toSet()	map((k,v)=>MapEntry())
Filter elements	where(f).toList()	where(f).toSet()	entries.where(f) then fromEntries
Sort	sort() -- in-place O(nlogn)	SplayTreeSet (sorted)	Sort entries List then fromEntries
Aggregate	reduce() / fold()	reduce() / fold()	Via .values or .entries iterable
Count elements	length -- O(1)	length -- O(1)	length -- O(1)
Empty check	isEmpty / isNotEmpty	isEmpty / isNotEmpty	isEmpty / isNotEmpty
To other type	toSet() / join()	toList() / join()	keys / values / entries
Unmodifiable view	List.unmodifiable(lst)	no direct method	Map.unmodifiable(map)
Fixed size	List.filled(n, v)	N/A	N/A
Sorted variant	sort() on List	SplayTreeSet (dart:collection)	SplayTreeMap (dart:collection)
Ordered variant	List (always ordered)	LinkedHashSet (insert order)	LinkedHashMap (default)
Literal syntax	[1, 2, 3]	{1, 2, 3}	{'key': value}
Empty literal	[]	<Type>{} (not {})	{}

Choosing the Right Collection

List when order matters or you need index access. Set when uniqueness matters or you need fast membership testing. Map when you look up values by a key. For sorted collections use SplayTreeSet/SplayTreeMap from dart:collection. For ordered+fast use LinkedHashMap/LinkedHashSet.